

ARM PrimeCell™
Single Master DMA Controller
(PL081)
Technical Reference Manual



ARM PrimeCell™

Single Master DMA Controller (PL081)

Technical Reference Manual

Copyright © 2001 ARM Limited. All rights reserved.

Release Information

The following changes have been made to this document.

Change history

Date	Issue	Change
May 23rd 2001	A	First draft

Proprietary Notice

Words and logos marked with © or ™ are registered trademarks or trademarks owned by ARM Limited. Other brands and names mentioned herein may be the trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM in good faith. However, all warranties implied or expressed, including but not limited to implied warranties of merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Confidentiality Status

This document is Open Access. This document has no restriction on distribution.

Product Status

The information in this document is final (information on a developed product).

Web Address

<http://www.arm.com>

Contents

ARM PrimeCell Single Master DMA Controller (PL081) Technical Reference Manual

Preface

About this document	viii
Further reading	x
Feedback	xi

Chapter 1

Introduction

1.1 About the ARM PrimeCell SMDMAC (PL081)	1-2
--	-----

Chapter 2

Functional Overview

2.1 PrimeCell SMDMAC functional description	2-2
2.2 System considerations	2-7
2.3 System connectivity	2-8

Chapter 3

Programmer's Model

3.1 About the programmer's model	3-2
3.2 Programming the PrimeCell SMDMAC	3-3
3.3 Summary of PrimeCell SMDMAC registers	3-6
3.4 Register descriptions	3-10
3.5 Address generation	3-33
3.6 Scatter/gather	3-34

3.7	Interrupt requests	3-36
3.8	PrimeCell SMDMAC data flow	3-39

Chapter 4

Programmer's Model for Test

4.1	PrimeCell SMDMAC test harness overview	4-2
4.2	Scan testing	4-3
4.3	Test registers	4-4
4.4	Integration test	4-7

Appendix A

ARM PrimeCell SMDMAC (PL081) Signal Descriptions

A.1	DMA interrupt request signals	A-2
A.2	DMA request and response signals	A-3
A.3	AHB slave signals	A-4
A.4	AHB master signals	A-6
A.5	AHB master bus request signals	A-8
A.6	Scan test control signals	A-9

Appendix B

DMA Interface

B.1	DMA request signals	B-2
B.2	DMA response signals	B-3
B.3	Flow control	B-4
B.4	Transfer types	B-5
B.5	Signal timing	B-21
B.6	Functional timing diagram	B-22
B.7	PrimeCell SMDMAC transfer timing diagram	B-23

Appendix C

Scatter/Gather

C.1	Scatter/gather through linked list operation	C-2
-----	--	-----

Preface

This preface introduces the ARM PrimeCell Single Master DMA Controller (PL081) and its reference documentation. It contains the following sections:

- *About this document* on page viii
- *Further reading* on page x
- *Feedback* on page xi.

About this document

This document is a technical reference manual for the ARM PrimeCell Single Master DMA Controller (SMDMA).

Intended audience

This document has been written for hardware and software engineers implementing System-on-Chip designs. It provides information to enable designers to integrate the peripheral into a target system as quickly as possible.

Using this manual

This document is organized into the following chapters:

Chapter 1 *Introduction*

Read this chapter for an introduction to the ARM PrimeCell SMDMAC.

Chapter 2 *Functional Overview*

Read this chapter for a description of the major functional blocks of the PrimeCell SMDMAC.

Chapter 3 *Programmer's Model*

Read this chapter for a description of the PrimeCell SMDMAC registers and programming details.

Chapter 4 *Programmer's Model for Test*

Read this chapter for an description of the logic in the PrimeCell SMDMAC for functional verification and production testing.

Appendix A *ARM PrimeCell SMDMAC (PL081) Signal Descriptions*

Read this appendix for details of the PrimeCell SMDMAC signals.

Appendix B *DMA Interface*

Read this appendix for details of the PrimeCell SMDMAC signals.

Typographical conventions

The following typographical conventions are used in this document:

bold	Highlights ARM processor signal names, and interface elements such as menu names. Also used for terms in descriptive lists, where appropriate.
<i>italic</i>	Highlights special terminology, cross-references, and citations.

<code>typewriter</code>	Denotes text that can be entered at the keyboard, such as commands, file names and program names, and source code.
<u><code>typewriter</code></u>	Denotes a permitted abbreviation for a command or option. The underlined text can be entered instead of the full command or option name.
<i><code>typewriter</code></i>	Denotes arguments to commands or functions where the argument is to be replaced by a specific value.
title bold	Denotes language keywords when used outside example code.

Timing diagram conventions

This manual contains one or more timing diagrams. Figure P-1 explains the components used in these diagrams. Any variations are clearly labeled when they occur. Therefore, no additional meaning must be attached unless specifically stated.

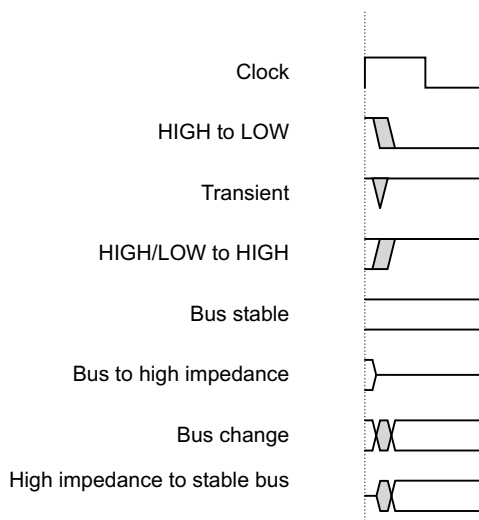


Figure P-1 Key to timing diagram conventions

Shaded bus and signal areas are undefined, so the bus or signal can assume any value within the shaded area at that time. The actual level is unimportant and does not affect normal operation.

Further reading

This section lists publications by ARM Limited, and by third parties.

ARM periodically provides updates and corrections to its documentation. See <http://www.arm.com> for current errata sheets and addenda.

See also the ARM Frequently Asked Questions list at:
<http://www.arm.com/armtech/PrimeCell?OpenDocument>

ARM publications

This document contains information that is specific to the ARM PrimeCell SMDMAC (PL081). Refer to the following documents for other relevant information:

- *AMBA Specification (Rev 2.0)* (ARM IHI 0011)
- *ARM PrimeCell Single Master DMA Controller (PL081) Design Manual* (PL081 DDES 0000)
- *ARM PrimeCell Single Master DMA Controller (PL081) Integration Manual* (PL081 INTM 0000).

Feedback

ARM Limited welcomes feedback both on the ARM PrimeCell SMDMAC (PL081), and on the documentation.

Feedback on the ARM PrimeCell SMDMAC (PL081)

If you have any comments or suggestions about this product, please contact your supplier giving:

- the product name
- a concise explanation of your comments.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- the document title
- the document number
- the page number(s) to which your comments refer
- a concise explanation of your comments.

General suggestions for additions and improvements are also welcome.

Chapter 1

Introduction

This chapter introduces the ARM PrimeCell Single Master DMA Controller (PL081). It contains the following section:

- *About the ARM PrimeCell SMDMAC (PL081) on page 1-2.*

1.1 About the ARM PrimeCell SMDMAC (PL081)

The PrimeCell Single Master DMA Controller (SMDMAC) is an *Advanced Microcontroller Bus Architecture* (AMBA) compliant *System-on-Chip* (SoC) peripheral that is developed, tested, and licensed by ARM.

The PrimeCell SMDMAC is an AMBA AHB module, and connects to the *Advanced High-performance Bus* (AHB).

The features of the PrimeCell SMDMAC are covered under *Features of the PrimeCell SMDMAC* on page 1-2.

1.1.1 Features of the PrimeCell SMDMAC

The PrimeCell SMDMAC offers:

- Compliance to the AMBA Specification (Rev 2.0) onwards for easy integration into SoC implementation.
- Two DMA channels. Each channel can support a unidirectional transfer.
- 16 DMA requests. The PrimeCell SMDMAC provides 16 peripheral DMA request lines.
- Single DMA and burst DMA request signals. Each peripheral connected to the PrimeCell SMDMAC can assert either a burst DMA request or a single DMA request. The DMA burst size is set by programming the PrimeCell SMDMAC.
- Memory-to-memory, memory-to-peripheral, peripheral-to-memory, and peripheral-to-peripheral transfers.
- Scatter or gather DMA is supported through the use of linked lists. This means that the source and destination areas do not need to occupy contiguous areas of memory.
- Hardware DMA channel priority. Each DMA channel has a specific hardware priority. DMA channel 0 has the highest priority and channel 1 has the lowest priority. If requests from two channels become active at the same time the channel with the highest priority is serviced first.
- AHB slave DMA programming interface. The PrimeCell SMDMAC is programmed by writing to the DMA control registers over the AHB slave interface.
- One AHB bus master for transferring data. This interface is used to transfer data when a DMA request goes active.

- 32-bit AHB master bus width.
- Incrementing or non-incrementing addressing for source and destination.
- Programmable DMA burst size. The DMA burst size can be programmed to more efficiently transfer data. Usually the burst size is set to half the size of the FIFO in the peripheral.
- Internal four word FIFO per channel.
- Supports 8, 16, and 32-bit wide transactions.
- Big-endian and little-endian support. The PrimeCell SMDMAC defaults to little-endian mode on reset.
- Separate and combined DMA error and DMA count interrupt requests. An interrupt to the processor can be generated on a DMA error or when a DMA count has reached 0 (this is usually used to indicate that a transfer has finished). Three interrupt request signals are used to do this:
 - **DMACINTTC** is used to signal when a transfer has completed.
 - **DMACINTERROR** is used to signal when an error has occurred.
 - **DMACINTCOMBINE** combines both the **DMACINTTC** and **DMACINTERROR** interrupt request signals. The **DMACINTCOMBINE** interrupt request can be used in systems, which have few interrupt controller request inputs.
- Interrupt masking. The DMA error and DMA terminal count interrupt requests can be masked.
- Raw interrupt status. The DMA error and DMA count raw interrupt status can be read prior to masking.
- Test registers for use in block and integration system level testing.
- Identification registers that uniquely identify the PrimeCell SMDMAC. These can be used by an operating system to automatically configure itself.

Chapter 2

Functional Overview

This chapter describes the major functional blocks of the ARM PrimeCell Single Master DMA Controller (PL081). It contains the following sections:

- *PrimeCell SMDMAC functional description* on page 2-2
- *System considerations* on page 2-7
- *System connectivity* on page 2-8.

2.1 PrimeCell SMDMAC functional description

The PrimeCell SMDMAC allows peripheral-to-memory, memory-to-peripheral, peripheral-to-peripheral, and memory-to-memory transactions.

Each DMA stream provides unidirectional serial DMA transfers for a single source and destination. For example, a bidirectional port requires one stream for transmit and one for receive. The source and destination areas can each be either a memory region or a peripheral, and can be accessed through the AHB master.

Figure 2-1 on page 2-3 shows a block diagram of the PrimeCell SMDMAC.

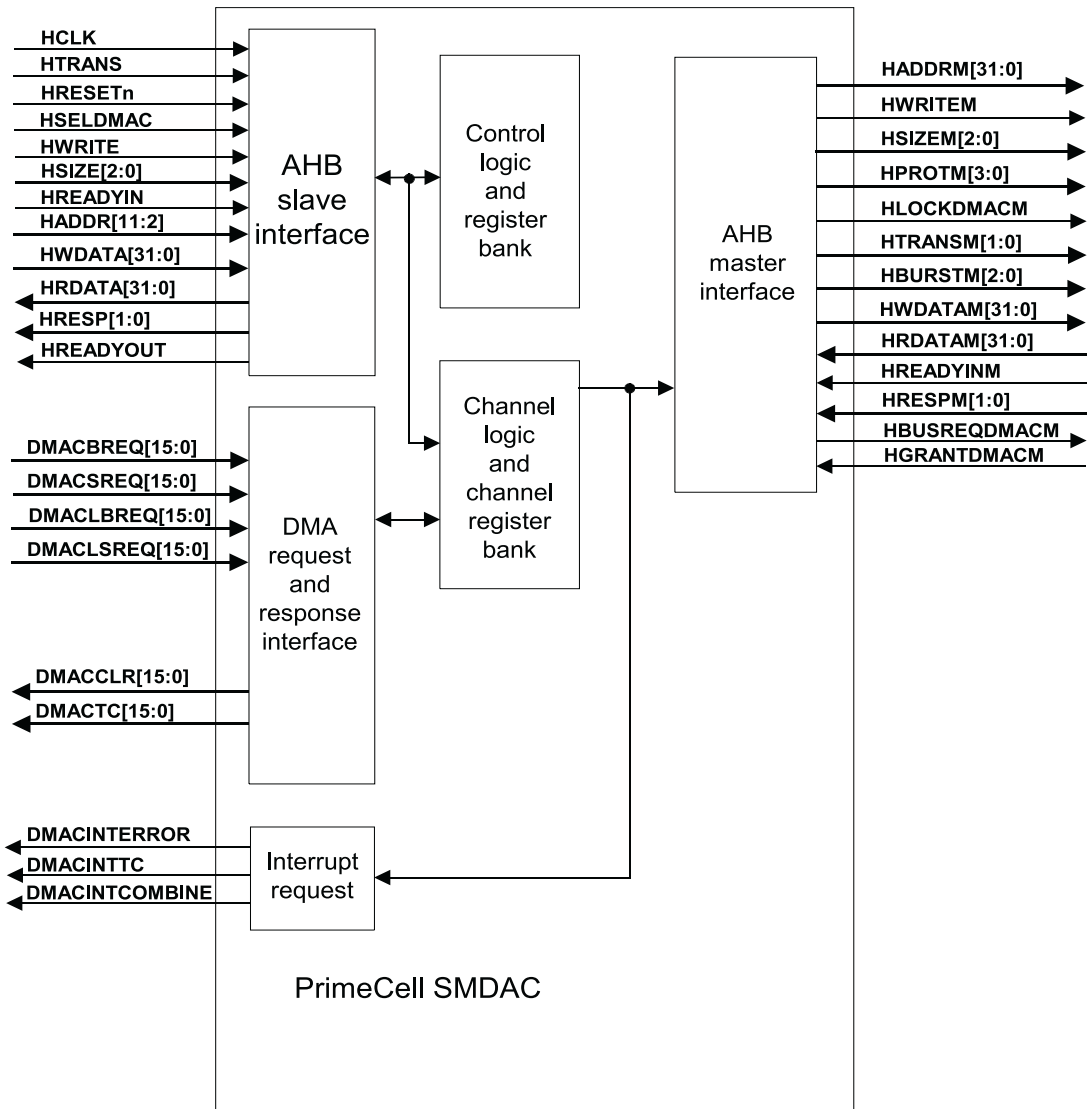


Figure 2-1 PrimeCell SMDMAC block diagram

Note

Test logic is not shown for clarity.

The functions of the PrimeCell SMDMAC are described in the following sections:

- *AHB slave interface* on page 2-4
- *Control logic and register bank* on page 2-4
- *DMA request and response interface* on page 2-4
- *Channel logic and channel register bank* on page 2-4
- *Interrupt request* on page 2-4
- *AHB master interface* on page 2-4
- *Channel hardware* on page 2-6
- *Test registers* on page 2-6
- *PrimeCell DMA request priority* on page 2-6

2.1.1 AHB slave interface

All transactions on the AHB slave programming bus of the PrimeCell SMDMAC are 32-bit wide. This eliminates endian issues when programming the PrimeCell SMDMAC.

2.1.2 Control logic and register bank

The register block stores data written, or to be read across the AMBA AHB interface. This block is used to program the PrimeCell SMDMAC using an AMBA AHB slave interface.

2.1.3 DMA request and response interface

See Appendix B *DMA Interface* for information on the DMA request and response interface.

2.1.4 Channel logic and channel register bank

The channel logic and channel register bank contains registers and logic required for each DMA channel.

2.1.5 Interrupt request

The interrupt request is used to generate interrupts to the ARM processor.

2.1.6 AHB master interface

The PrimeCell SMDMAC contains a full AHB master, see Figure 2-2 on page 2-5 for a block diagram showing the master connected into a system.

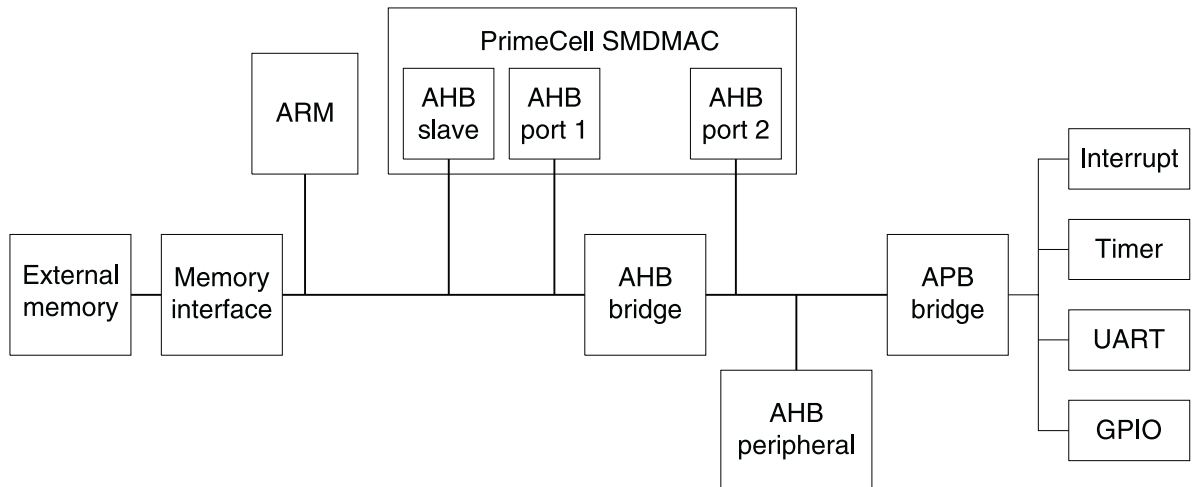


Figure 2-2 AHB master

The AHB master is capable of dealing with all types of AHB transactions, including:

- Split, retry, and error responses from slaves. If a peripheral performs a split or retry, the PrimeCell SMDMAC stalls and waits until the transaction can complete.
- Locked transfers for source and destination of each stream.
- Setting of protection bits for transfers on each stream.

All AHB signals are connected as defined in the AHB specification. The AHB buses are required to be synchronous, in other words they must use the same **HCLK**. Support for asynchronous AHB buses is not defined within the PrimeCell SMDMAC, and must be implemented through the use of wrappers, if required.

Bus and transfer widths

The default width for the AHB master is a 32-bit bus. Source and destination transfers can be of differing widths, and can be the same width or narrower than the physical bus width. The PrimeCell SMDMAC packs or unpacks data as appropriate. The PrimeCell SMDMAC uses **HSIZE** to indicate the width of a transfer, and if this fails to match the width expected by the peripheral, then the peripheral can assert an error on **HRESP**.

Endianness

The PrimeCell SMDMAC can cope with both little-endian and big-endian addressing.

Error conditions

An error during a DMA transfer is flagged directly by the peripheral by asserting an Error response on the AHB bus during the transfer. The PrimeCell SMDMAC automatically disables the DMA stream after the current transfer has completed, and can optionally generate an error interrupt to the CPU. This error interrupt can be masked.

2.1.7 Channel hardware

Each stream is supported by a dedicated hardware channel, including source and destination controllers, and a FIFO. This allows for better latency than a DMA controller with only a single hardware channel shared between several DMA streams and simplifies the control logic.

2.1.8 Test registers

Test registers are provided for integration testing.

Test registers must not be read or written to during normal use.

The integration testing verifies that the PrimeCell SMDMAC has been connected into a system correctly, allowing each input and output to be both written to and read.

2.1.9 PrimeCell DMA request priority

PrimeCell DMA channel priority is fixed, DMA channel 0 has the highest priority and DMA channel 1 has the lowest priority.

If the PrimeCell SMDMAC is transferring data for the lower priority channel and then the higher priority channel goes active, it completes the number of transfers delegated to the master interface by the lower priority channel before switching over to transfer data for the higher priority channel. In the worst case this is as large as a one quad word.

Channel 1 in the PrimeCell SMDMAC is designed so that it cannot saturate the AHB bus. If it goes active, the PrimeCell SMDMAC relinquishes control of the bus (for a bus cycle), after four transfers of the programmed size (irrespective of the size of transfer). This allows other AHB masters to access the bus.

It is recommended that memory-to-memory transactions use the low priority channel. Otherwise other (lower priority) AHB bus masters are prevented from accessing the bus during PrimeCell SMDMAC memory-to-memory transfer.

2.2 System considerations

Reducing the number of transactions that occur on the buses reduces the latency on the bus, improves system performance, and reduces power consumption. Therefore, the following design considerations are recommended:

- All memory transactions are (in the standard configuration) 32 bits wide to improve bus efficiency.
- Peripherals whose natural word size is less than 32 bits must contain byte or halfword packing hardware so that all transactions can be made 32 bits wide.
- Slow peripherals that normally use wait states must contain FIFOs so that data can be transferred at full speed using burst transfers.

2.3 System connectivity

Figure 2-3 shows how the PrimeCell SMDMAC is connected to a system.

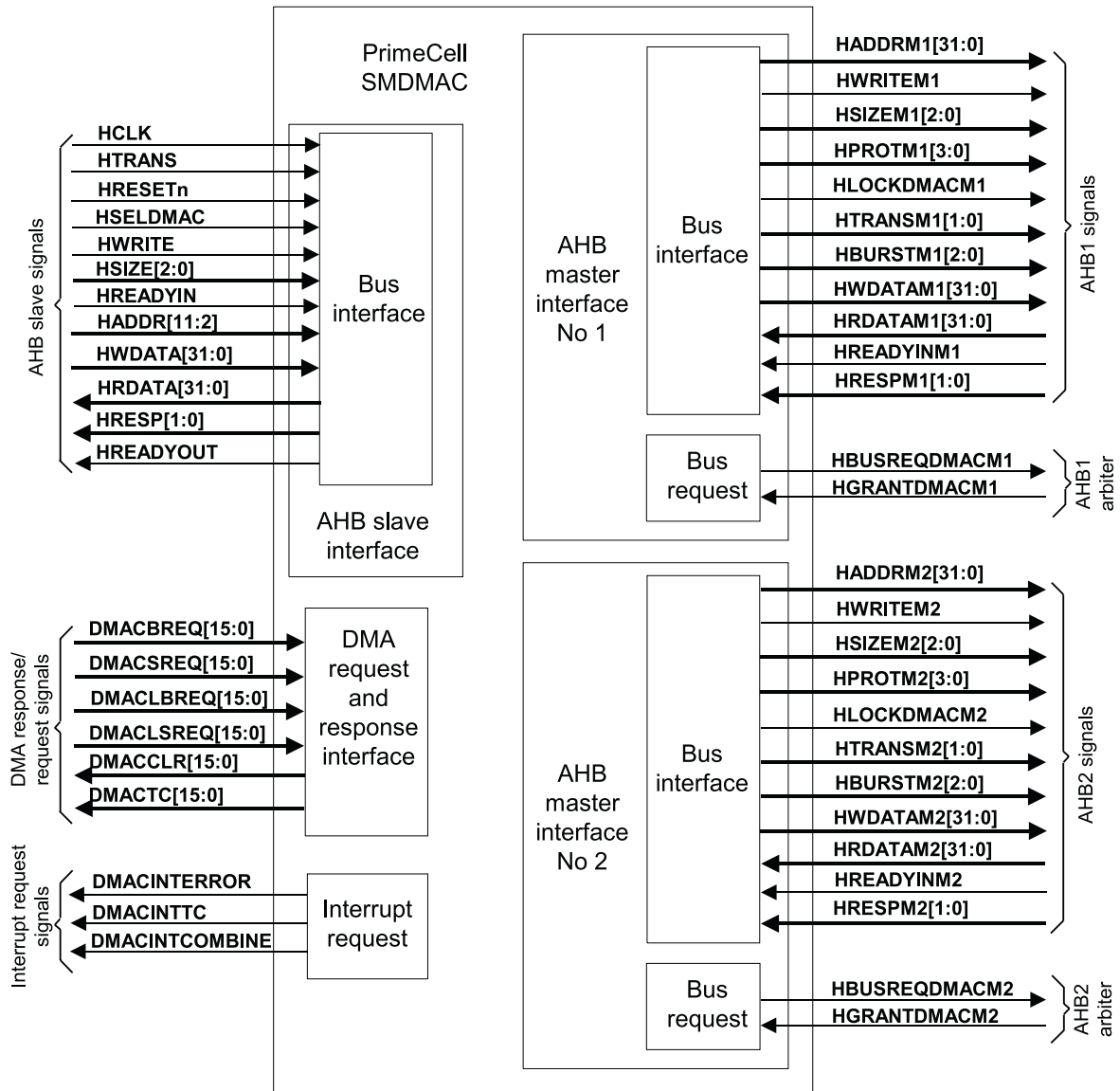


Figure 2-3 PrimeCell SMDMAC connectivity

2.3.1 AHB interface

The AHB slave and master interface executes from **HCLK**.

2.3.2 AHB slave interface

The AHB slave interface is used to program the PrimeCell SMDMAC. The port level connections of AHB slave interface module are shown in Figure 2-3 on page 2-8.

2.3.3 AHB master interface

Unless otherwise stated this interface must be connected as described in the *AMBA Specification (2.0)*.

The various AHB signals can be set while performing DMA transfers.

Protection control

The **HPROT[3:0]** bits are programmed by software for each PrimeCell DMA channel. The bits are set:

- **HPROT[0]**, opcode, or data. This bit is hardcoded to Data, 1.
- **HPROT[1]**, User or privileged. User = 0, privileged = 1. Programmed by software see *Channel control registers, DMACCxControl* on page 3-19. During *Linked List Item (LLI)* loads **HPROT[1]** is made 1 (privileged).
- **HPROT[2]**, bufferable or nonbufferable. Nonbufferable = 0, bufferable = 1. Programmed by software see *Channel control registers, DMACCxControl* on page 3-19. During LLI loads **HPROT[2]** is made 0.
- **HPROT[3]**, cacheable or noncacheable. Not cacheable = 0, cacheable = 1. Programmed by software see *Channel control registers, DMACCxControl* on page 3-19. During LLI loads **HPROT[3]** is made 1.

Peripherals can interpret the **HPROT** information as required to help perform efficient transactions. For example:

- The **HPROT[1]** User or privileged bit can be used to protect certain peripherals or memory spaces from User mode transactions.
- The **HPROT[2]** bufferable or nonbufferable bit can be used to indicate to an AMBA bridge that the write can complete in zero wait states on the source bus. This is without waiting for it to arbitrate for the destination bus and for the slave to accept the data.

- The **HPROT[3]** cacheable or noncacheable bit can be used by an AMBA bridge so that on the first read of a burst of eight it could transfer the whole burst of eight reads on the destination bus, rather than pass the transactions through one at a time.

Lock control

Set the lock bit by programming the relevant bit in the DMA channel.

Bus width

The **HSIZE[1:0]** bits are programmed by the source width (SWidth) or destination width (DWidth) values in the **DMACCxControl** register.

2.3.4 Interrupt generation logic

Individual maskable active HIGH interrupts are generated by the PrimeCell SMDMAC. A combined interrupt output is also generated as an OR function of the individual interrupt requests.

You can use the single combined interrupt with a system interrupt controller that provides another level of masking on a per-peripheral basis. This allows you to use modular device drivers that always know where to find the interrupt source control register bits.

You can also use the individual interrupt requests with a system interrupt controller that provides masking for the outputs of each peripheral. In this way, a global interrupt service routine can read the entire set of sources from one wide register in the system interrupt controller. This is useful where the time to read from the peripheral registers is significant compared to the CPU clock speed in a real-time system.

The peripheral supports both these methods.

2.3.5 Interrupt controller connectivity

The interrupt request signals of the PrimeCell SMDMAC can be connected to an interrupt controller in one of two ways. For higher performance systems the **DMACINTERR** and **DMACINTTC** interrupt request signals must be connected to the interrupt controller. For lower performance systems where the interrupt controller has fewer interrupt request input lines, the **DMACINTR** interrupt request signal can be used. For further information see *Interrupt requests* on page 3-36. Figure 2-4 on page 2-11 and Figure 2-5 on page 2-11 show connections to higher and lower performance systems respectively.

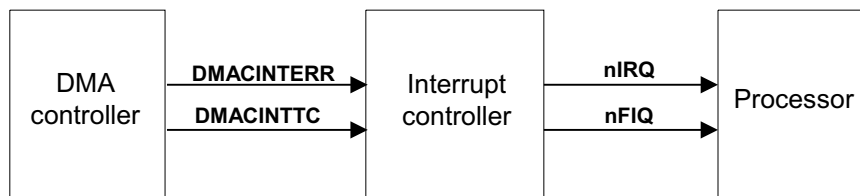


Figure 2-4 Connection for higher performance systems

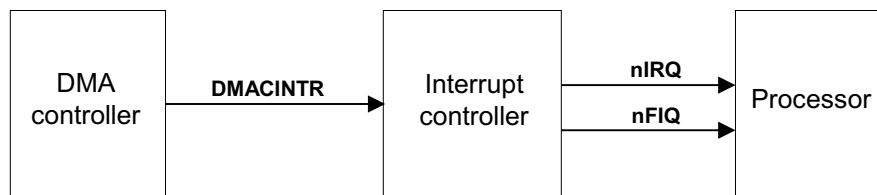


Figure 2-5 Connection for lower performance systems

2.3.6 DMA request and response connectivity

The diagram shows how the DMA request and response signals can be connected to a peripheral. However some peripherals do not make use of all of these signals. Output signals that are not required can be left unconnected, input signals that are not required must be tied LOW.

See Appendix B *DMA Interface* for further information on the DMA request and response interface. Figure 2-6 shows a example of peripheral that makes use of all the DMA request and grant signals.

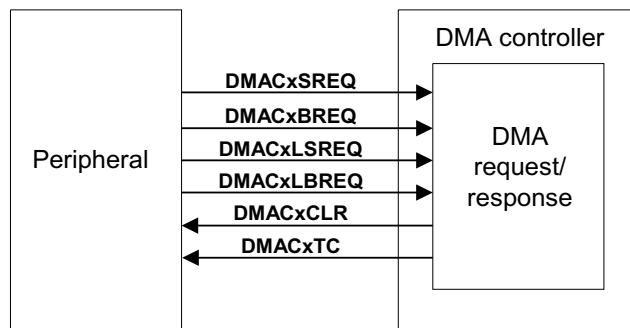


Figure 2-6 Complex example of connectivity

Figure 2-7 shows a simple example of connectivity.

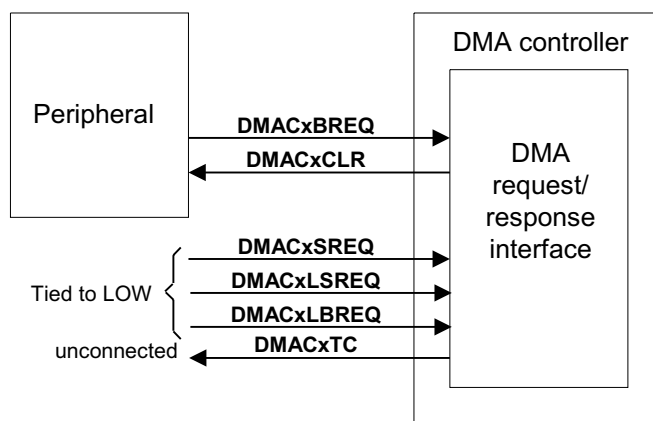


Figure 2-7 Simple example of connectivity

Chapter 3

Programmer's Model

This chapter describes the ARM PrimeCell Single Master DMA Controller (PL081) registers and provides details required when programming the microcontroller. It contains the following sections:

- *About the programmer's model* on page 3-2
- *Programming the PrimeCell SMDMAC* on page 3-3
- *Summary of PrimeCell SMDMAC registers* on page 3-6
- *Register descriptions* on page 3-10
- *Address generation* on page 3-33
- *Scatter/gather* on page 3-34
- *Interrupt requests* on page 3-36
- *PrimeCell SMDMAC data flow* on page 3-39.

3.1 About the programmer's model

The PrimeCell SMDMAC allows peripheral-to-memory, memory-to-peripheral, peripheral-to-peripheral, and memory-to-memory transactions.

Each DMA stream is configured to provide unidirectional DMA transfers for a single source and destination. For example, a bidirectional serial port requires one stream for transmit and one for receive. The source and destination areas can each be either a memory region or a peripheral, and can be accessed through the same AHB master, or one area by each master.

The base address of the PrimeCell SMDMAC is not fixed, and can be different for any particular system implementation. However, the offset of any particular register from the base address is fixed.

3.1.1 Register fields

The following applies to the registers used in the Primecell DMAC:

- Reserved or unused address locations must not be accessed as this can result in unpredictable behavior of the device.
- Reserved or unused bits of registers must be written as zero, and ignored on read unless otherwise stated in the relevant text.
- All registers bits are reset to a logic 0 by a system or power on reset unless otherwise stated in the relevant text.
- Unless otherwise stated in the relevant text, all registers support read and write accesses. A write updates the contents of a register and a read returns the contents of the register.
- All registers defined in this document can only be accessed using word reads and word writes, unless otherwise stated in the relevant text.

3.2 Programming the PrimeCell SMDMAC

All transactions on the AHB Slave programming bus must be 32-bit wide. This eliminates endian issues when programming the SMDMAC.

3.2.1 Enabling the PrimeCell SMDMAC

To enable the PrimeCell SMDMAC set the DMA Enable bit in the DMACConfiguration register.

3.2.2 Disabling the PrimeCell SMDMAC

To disable the PrimeCell SMDMAC:

1. Read the DMACEnbldChns register and ensure that all the DMA channels have been disabled. If any channels are active see *Disabling a DMA channel* on page 3-3.
2. Disable the PrimeCell SMDMAC by writing 0 to the DMA Enable bit in the DMACConfiguration register.

3.2.3 Enabling a DMA channel

To enable the DMA channel set the Channel Enable bit in the relevant DMA channel configuration register.

———— **Note** ————

The channel must be fully initialized before it is enabled. Additionally, the Enable bit of the PrimeCell SMDMAC must be set before any channels are enabled.

3.2.4 Disabling a DMA channel

A DMA channel can be disabled in three ways:

- Write directly to the Channel Enable bit. Any outstanding data in the FIFOs is lost if this method is used.
- Use the Active and Halt bits in conjunction with the Channel Enable bit.
- Wait until the transfer completes. The channel is then automatically disabled.

Disabling a DMA channel and losing data in the FIFO

Clear the relevant Channel Enable bit in the relevant channel configuration register. The current AHB transfer (if one is in progress) completes and the channel is disabled. Any data in the FIFO is lost.

Disabling a DMA channel without losing data in the FIFO

To disabling a DMA channel without losing data in the FIFO:

1. Set the Halt bit in the relevant channel configuration register. This causes any further DMA requests to be ignored.
2. Poll the Active bit in the relevant channel configuration register until it reaches 0. This bit indicates whether there is any data in the channel which has to be transferred.
3. Clear the Channel Enable bit in the relevant channel configuration register.

3.2.5 Set up a new DMA transfer

To set up a new DMA transfer:

1. If the channel is not set aside for the DMA transaction:
 - a. Read the DMACEnbldChns controller register and find out which channels are inactive.
 - b. Choose an inactive channel which has the required priority.
2. Program the PrimeCell SMDMAC.

3.2.6 Halting a DMA channel

Set the Halt bit in the relevant DMA channel configuration register. The current source request is serviced. Any further source DMA requests are ignored until the Halt bit is cleared.

3.2.7 Programming a DMA channel

To program a DMA channel:

1. Choose a free DMA channel with the priority needed. Where DMA channel 0 has the highest priority and DMA channel 7 the lowest priority.

2. Clear any pending interrupts on the channel to be used by writing to the DMACIntTCClr and DMACIntErrClr registers. The previous channel operation might have left interrupts active.
3. Write the source address into the DMACCxSrcAddr register.
4. Write the destination address into the DMACCxDestAddr register.
5. Write the address of the next LLI into the DMACCxLLI register. If the transfer comprises of a single packet of data then 0 must be written into this register.
6. Write the control information into the DMACCxControl register.
7. Write the channel configuration information into the DMACCxConfiguration register. If the Enable bit is set then the DMA channel is automatically enabled.

3.3 Summary of PrimeCell SMDMAC registers

The PrimeCell SMDMAC registers are shown in Table 3-1.

Table 3-1 PrimeCell SMDMAC register summary

Address	Type	Width	Reset value	Name	Description
SMDMAC base + 0x000	Read	2	0x0	DMACIntStatus	This register provides the interrupt status of the PrimeCell SMDMAC. A HIGH bit indicates that a specific DMA channel interrupt is active.
SMDMAC base + 0x004	Read	2	0x0	DMACIntTCStatus	This register is used to determine whether an interrupt was generated due to the transaction completing (terminal count). A HIGH bit indicates that the transaction completed.
SMDMAC base + 0x008	Write	2	-	DMACIntTCClear	When writing to this register, each data bit that is HIGH causes the corresponding bit in the DMACIntTCStatus and DMACRawIntTCStatus registers to be cleared. Data bits that are LOW have no effect on the corresponding bit in the register.
SMDMAC base + 0x00C	Read	2	0x0	DMACIntErrorStatus	This register is used to determine whether an interrupt was generated due to an error being generated.
SMDMAC base + 0x010	Write	2	-	DMACIntErrClr	When writing to this register, each data bit that is HIGH causes the corresponding bit in the DMACIntErrorStatus and DMACRawIntErrorStatus registers to be cleared. Data bits that are LOW have no effect on the corresponding bit in the register.

Table 3-1 PrimeCell SMDMAC register summary (continued)

Address	Type	Width	Reset value	Name	Description
SMDMAC base + 0x014	Read	2	-	DMACRawIntTCStatus	This register provides the raw status of DMA terminal count interrupts prior to masking. A HIGH bit indicates that the interrupt request is active prior to masking.
SMDMAC base + 0x018	Read	2	-	DMACRawIntErrorStatus	This register provides the raw status of DMA error interrupts prior to masking. A HIGH bit indicates that the interrupt request is active prior to masking.
SMDMAC base + 0x01C	Read	2	0x0	DMACEnbldChns	This register shows which DMA channels are enabled. A HIGH bit indicates that a DMA channel is enabled.
SMDMAC base + 0x020	Read/write	16	0x0000	DMACSoftBReq	This register allows DMA burst requests to be generated by software.
SMDMAC base + 0x024	Read/write	16	0x0000	DMACSoftSReq	This register allows DMA single requests to be generated by software.
SMDMAC base + 0x028	Read/write	16	0x0000	DMACSoftLBReq	This register allows DMA last burst requests to be generated by software.
SMDMAC base + 0x02C	Read/write	16	0x0000	DMACSoftLSReq	This register enables SMDMAC DMA last single requests to be generated by software.
SMDMAC base + 0x030	Read/write	2	0b000	DMACConfiguration	This register is used to configure the PrimeCell SMDMAC.
SMDMAC base + 0x34	Read/write	16	0x0000	DMACSync	This register enables or disables synchronization logic for the DMA request signals.

Table 3-1 PrimeCell SMDMAC register summary (continued)

Address	Type	Width	Reset value	Name	Description
SMDMAC base + 0x100	Read/write	32	0x00000000	DMACC0SrcAddr	DMA channel 0 source address.
SMDMAC base + 0x104	Read/write	32	0x00000000	DMACC0DestAddr	DMA channel 0 destination address.
SMDMAC base + 0x108	Read/write	31	0x00000000	DMACC0LLI	DMA channel 0 linked list address.
SMDMAC base + 0x10C	Read/write	30	0x00000000	DMACC0Control	DMA channel 0 control.
SMDMAC base + 0x110	Read/write	19	0x000000	DMACC0Configuration	DMA channel 0 configuration register.
SMDMAC base + 0x120	Read/write	32	0x00000000	DMACC1SrcAddr	DMA channel 1 source address.
SMDMAC base + 0x124	Read/write	32	0x00000000	DMACC1DestAddr	DMA channel 1 destination address.
SMDMAC base + 0x128	Read/write	31	0x00000000	DMACC1LLI	DMA channel 1 linked list address.
SMDMAC base + 0x12C	Read/write	30	0x00000000	DMACC1Control	DMA channel 1 control.
SMDMAC base + 0x130	Read/write	19	0x000000	DMACC1Configuration	DMA channel 1 configuration register.
SMDMAC base + 0xFE0	Read	8	0x81	DMACPeriphID0	Peripheral identification register bits [7:0].
SMDMAC base + 0xFE4	Read	8	0x10	DMACPeriphID1	Peripheral identification register bits [15:8].
SMDMAC base + 0xFE8	Read	8	0x04	DMACPeriphID2	Peripheral identification register bits [23:16].
SMDMAC base + 0xFEC	Read	8	0x00	DMACPeriphID3	Peripheral identification register bits [31:24].
SMDMAC base + 0xFF0	Read	8	0x00	DMACPCellID0	PrimeCell identification register bits [7:0].

Table 3-1 PrimeCell SMDMAC register summary (continued)

Address	Type	Width	Reset value	Name	Description
SMDMAC base +0xFF4	Read	8	0xF0	DMACPCellID1	PrimeCell identification register bits [15:8].
SMDMAC base +0xFF8	Read	8	0x05	DMACPCellID2	PrimeCell identification register bits [23:16].
SMDMAC base + 0xFFC	Read	8	0xB1	DMACPCellID3	PrimeCell identification register bits [31:24].

3.4 Register descriptions

The following PrimeCell SMDMAC registers are described in this section:

- *Interrupt status register, DMACIntStatus* on page 3-10
- *Interrupt terminal count status register, DMACIntTCStatus* on page 3-11
- *Interrupt terminal count clear register, DMACIntTCClear* on page 3-11
- *Interrupt error status register, DMACIntErrorStatus* on page 3-11
- *Interrupt error clear register, DMACIntErrClr* on page 3-12
- *Raw interrupt terminal count status register, DMACRawIntTCStatus* on page 3-12
- *Raw error interrupt status register, DMACRawIntErrorStatus* on page 3-13
- *Enabled channel register, DMACEnbldChns* on page 3-13
- *Software burst request register, DMACSoftBReq* on page 3-13
- *Software single request register, DMACSoftSReq* on page 3-14
- *Software last burst request register, DMACSoftLBReq* on page 3-14
- *Software last single request register, DMACSoftLSReq* on page 3-15
- *Configuration register, DMACConfiguration* on page 3-15
- *Synchronization register, DMACSync* on page 3-16
- *Channel source address registers, DMACCxSrcAddr* on page 3-17
- *Channel destination address registers, DMACCxDestAddr* on page 3-18
- *Channel linked list item register, DMACCxLLI* on page 3-18
- *Channel control registers, DMACCxControl* on page 3-19
- *Channel configuration registers, DMACCxConfiguration* on page 3-23
- *Peripheral identification registers, DMACPeriphID0-3* on page 3-26
- *PrimeCell identification registers, DMACPCellID0-3* on page 3-30.

3.4.1 Interrupt status register, DMACIntStatus

The DMACIntStatus register is read-only and shows the status of the interrupts after masking. A HIGH bit indicates that a specific DMA channel interrupt request is active. The request can be generated from either the error or terminal count interrupt requests. Table 3-2 shows the bit assignment of the DMACIntStatus register.

Table 3-2 DMACIntStatus register

Bits	Name	Type	Function
1:0	IntStatus	Read	Status of the DMA interrupts after masking

3.4.2 Interrupt terminal count status register, DMACIntTCStatus

The DMACIntTCStatus register is read-only and indicates the status of the terminal count after masking.

This register must be used in conjunction with the DMACIntStatus register if the combined interrupt request, DMACINTCOMBINE, is used to request interrupts.

If the DMACINTTC interrupt request is used then you only have to read the DMACIntTCStatus register to ascertain the source of the interrupt request.

Table 3-3 shows the bit assignment of the DMACIntTCStatus register.

Table 3-3 DMACIntTCStatus register

Bits	Name	Type	Function
1:0	IntTCStatus	Read	Interrupt terminal count request status

3.4.3 Interrupt terminal count clear register, DMACIntTCClear

The DMACIntTCClear register is write-only and is used to clear a terminal count interrupt request.

When writing to this register, each data bit that is set HIGH causes the corresponding bit in the status register to be cleared. Data bits that are LOW have no effect on the corresponding bit in the register.

Table 3-4 shows the bit assignment of the DMACIntTCClear register.

Table 3-4 DMACIntTCClear register

Bits	Name	Type	Function
1:0	IntTCClear	Write	Terminal count request clear

3.4.4 Interrupt error status register, DMACIntErrorStatus

The DMACIntErrorStatus register is read-only register and indicates the status of the error request after masking.

This register must be used in conjunction with the DMACIntStatus register if the combined interrupt request, DMACINTCOMBINE, is used to request interrupts.

If the DMACINTERRUPT interrupt request is used only the DMACIntErrorStatus register needs to be read. Table 3-5 shows the bit assignment of the DMACIntErrorStatus register.

Table 3-5 DMACIntErrorStatus register

Bits	Name	Type	Function
1:0	IntErrorStatus	Read	Interrupt error status

3.4.5 Interrupt error clear register, DMACIntErrClr

The DMACIntErrClr register is a write-only register and is used to clear the error interrupt requests. When writing to this register, each data bit that is HIGH causes the corresponding bit in the status register to be cleared. Data bits that are LOW have no effect on the corresponding bit in the register. Table 3-6 shows the bit assignment of the DMACIntErrClr register.

Table 3-6 DMACIntErrClr register

Bits	Name	Type	Function
1:0	IntErrClr	Write	Interrupt error clear

3.4.6 Raw interrupt terminal count status register, DMACRawIntTCStatus

The DMACRawIntTCStatus register is read-only. It indicates which DMA channel is requesting a transfer complete (terminal count interrupt) prior to masking. A HIGH bit indicates that the terminal count interrupt request is active prior to masking.

Table 3-7 shows the bit assignment of the DMACRawIntTCStatus register.

Table 3-7 DMACRawIntTCStatus register

Bits	Name	Type	Function
1:0	RawIntTCStatus	Read	Status of the terminal count interrupt prior to masking

3.4.7 Raw error interrupt status register, DMACRawIntErrorStatus

The DMACRawIntErrorStatus register is read-only. It indicates which DMA channel is requesting an error interrupt prior to masking. A HIGH bit indicates that the error interrupt request is active prior to masking. Table 3-8 shows the bit assignment of register of the DMACRawIntErrorStatus register.

Table 3-8 DMACRawIntErrorStatus register

Bits	Name	Type	Function
1:0	RawIntErrorStatus	Read	Status of the error interrupt prior to masking

3.4.8 Enabled channel register, DMACEnbldChns

The DMACEnbldChns register is read-only and indicates which DMA channel is enabled, as indicated by the Enable bit in the DMACCxConfiguration register. A HIGH bit indicates that a DMA channel is enabled. A bit is cleared on completion of the DMA transfer.

Table 3-9 shows the bit assignment of the DMACEnbldChns register.

Table 3-9 DMACEnbldChns register

Bits	Name	Type	Function
1:0	EnabledChannels	Read	Channel enable status

3.4.9 Software burst request register, DMACSoftBReq

The DMACSoftBReq register is read/write and it allows DMA burst requests to be generated by software. A DMA request can be generated for each source by writing a 1 to the corresponding register bit. A register bit is cleared when the transaction has completed. Writing 0 to this register has no effect.

Reading the register indicates which sources are requesting DMA burst transfers. A request can be generated from either a peripheral or the software request register.

Table 3-10 shows the bit assignment of the DMACSoftBReq register.

Table 3-10 DMACSoftBReq register

Bits	Name	Type	Function
15:0	SoftBReq	Read/write	Software burst request

Note

It is recommended that software and hardware peripheral requests are not used at the same time.

3.4.10 Software single request register, DMACSoftSReq

The DMACSoftSReq read/write register allows DMA single requests to be generated by software. A DMA request can be generated for each source by writing a 1 to the corresponding register bit. A register bit is cleared when the transaction has completed. Writing 0 to this register has no effect.

Reading the register indicates which sources are requesting single DMA transfers. A request can be generated from either a peripheral or the software request register.

Table 3-11 shows the bit assignment of the DMACSoftSReq register.

Table 3-11 DMACSoftSReq register

Bits	Name	Type	Function
15:0	SoftSReq	Read/write	Software single request

Note

It is recommended that software and hardware peripheral requests are not used at the same time.

3.4.11 Software last burst request register, DMACSoftLBReq

The DMACSoftLBReq read/write register allows DMA last burst requests to be generated by software. A DMA request can be generated for each source by writing a 1 to the corresponding register bit. A register bit is cleared when the transaction has completed. Writing 0 to this register has no effect.

Reading the register indicates which sources are requesting last burst DMA transfers. A request can be generated from either a peripheral or the software request register.

Table 3-12 shows the bit assignment of the DMACSoftLBReq register.

Table 3-12 DMACSoftLBReq register

Bits	Name	Type	Function
15:0	SoftLBReq	Read/write	Software last burst request

3.4.12 Software last single request register, DMACSoftLSReq

The DMACSoftLSReq read/write register allows DMA last single requests to be generated by software. A DMA request can be generated for each source by writing a 1 to the corresponding register bit. A register bit is cleared when the transaction has completed. Writing 0 to this register has no effect.

Reading the register indicates which sources are requesting last single DMA transfers. A request can be generated from either a peripheral or the software request register.

Table 3-13 shows the bit assignment of the DMACSoftLSReq register.

Table 3-13 DMACSoftLSReq register

Bits	Name	Type	Function
15:0	SoftLSReq	Read/write	Software last single request

3.4.13 Configuration register, DMACConfiguration

The DMACConfiguration read/write register is used to configure the operation of the PrimeCell SMDMAC. The endianness of the AHB master interface can be altered by writing to the M bit of this register. The AHB master interface is set to little-endian mode on reset.

Table 3-14 shows the bit assignment of the DMACConfiguration register.

Table 3-14 DMACConfiguration register

Bits	Name	Type	Function
31:2	Reserved	-	Reserved, read as zero, do not modify.
1	M	Read/write	AHB Master endianness configuration: 0 = little-endian mode 1 = big-endian mode. This bit is reset to 0.
0	E	Read/write	PrimeCell SMDMAC enable: 0 = disabled 1 = enabled. This bit is reset to 0. Disabling the PrimeCell SMDMAC reduces power consumption.

3.4.14 Synchronization register, DMACSync

The DMACSync read/write register is used to enable or disable synchronization logic for the DMA request signals. The DMA request signals consist of the **DMACBREQ[15:0]**, **DMACSREQ[15:0]**, **DMACLBREQ[15:0]**, and **DMACLSREQ[15:0]** signals. A bit set to 0 enables the synchronization logic for a particular group of DMA requests. A bit set to 1 disables the synchronization logic for a particular group of DMA requests.

This register is reset to 0, synchronization logic enabled.

———— **Note** ————

Synchronization logic must be used when the peripheral generating the DMA request runs on a different clock to the SMDMAC. For peripherals running on the same clock as the SMDMAC disabling the synchronization logic improves the DMA request response time.

If necessary, the DMA response signals, **DMACCLR** and **DMACTC**, must be synchronized in the peripheral.

Table 3-15 shows the bit assignment of the DMACSync register.

Table 3-15 DMACSync register

Bits	Name	Type	Function
15:0	DMACSync	Read/write	DMA synchronization logic for DMA request signals enabled or disabled. A LOW bit indicates that the synchronization logic for the DMACBREQ[15:0] , DMACSREQ[15:0] , DMACLBREQ[15:0] , and DMACLSREQ[15:0] request signals is enabled. A HIGH bit indicates that the synchronization logic is disabled.

3.4.15 Channel registers

The channel registers are used to program a DMA channel. These registers consist of:

- two DMACCxSrcAddr registers
- two DMACCxDestAddr registers
- two DMACCxLLI registers
- two DMACCxControl registers
- two DMACCxConfiguration registers.

When performing scatter/gather DMA the first four registers are automatically updated.

3.4.16 Channel source address registers, DMACCxSrcAddr

The two read/write DMACCxSrcAddr registers contain the current source address (byte-aligned) of the data to be transferred.

Each register is programmed directly by software before the appropriate channel is enabled. When the DMA channel is enabled this register is updated:

- as the source address is incremented
- by following the linked list when a complete packet of data has been transferred.

Reading the register when the channel is active does not provide useful information. This is because by the time that software has processed the value read, the channel might have progressed. It is intended to be read only when the channel has stopped, in which case it shows the source address of the last item read.

———— **Note** —————

The source and destination addresses must be aligned to the source and destination widths.

Table 3-16 shows the bit assignment of the DMACCxSrcAddr registers.

Table 3-16 DMACCxSrcAddr registers

Bits	Name	Type	Function
31:0	SrcAddr	Read/write	DMA source address

3.4.17 Channel destination address registers, DMACCxDestAddr

The two read/write DMACCxDestAddr registers contain the current destination address (byte-aligned) of the data to be transferred.

Each register is programmed directly by software before the channel is enabled. When the DMA channel is enabled the register is updated as the destination address is incremented and by following the linked list when a complete packet of data has been transferred.

Reading the register when the channel is active does not provide useful information. This is because by the time that software has processed the value read, the channel might have progressed. It is intended to be read only when a channel has stopped, in which case it shows the destination address of the last item read.

Table 3-17 shows the bit assignment of a DMACCxDestAddr register.

Table 3-17 DMACCxDestAddr register

Bits	Name	Type	Function
31:0	DestAddr	Read/write	DMA destination address

3.4.18 Channel linked list item register, DMACCxLLI

The two read/write DMACCxLLI registers contain a word aligned address of the next *Linked List Item* (LLI). If the LLI is 0, then the current LLI is the last in the chain, and the DMA channel is disabled once all DMA transfers associated with it are completed.

———— **Note** ————

Programming this register when the DMA channel is enabled has unpredictable side effects.

Table 3-18 shows the bit assignment of a DMACCxLLI register.

Table 3-18 DMACCxLLI register

Bits	Name	Type	Function
31:2	LLI	Read/write	Linked list item. Bits [31:2] of the address for the next LLI. Address bits [1:0] are 0.
1	R	Read/write	Reserved, and must be written as 0, masked on read.
0	Reserved	-	Reserved, read as zero, do not modify.

Note

To make loading the LLIs more efficient for some systems, the LLI data structures can be made 4-word aligned.

3.4.19 Channel control registers, DMACCxControl

The two read/write DMACCxControl registers contain DMA channel control information such as the transfer size, burst size, and transfer width.

Each register is programmed directly by software before the DMA channel is enabled. When the channel is enabled the register is updated by following the linked list when a complete packet of data has been transferred.

Reading the register whilst the channel is active does not give useful information. This is because by the time that software has processed the value read, the channel might have progressed. It is intended to be read only when a channel has stopped.

Table 3-19 shows the bit assignment of a DMACCxControl register.

Table 3-19 DMACCxControl register

Bits	Name	Type	Function
31	I	Read/write	Terminal count interrupt enable bit. It controls whether the current LLI is expected to trigger the terminal count interrupt.
30:28	Prot	Read/write	Protection.
27	DI	Read/write	Destination increment. When set the destination address is incremented after each transfer.

Table 3-19 DMACCxControl register (continued)

Bits	Name	Type	Function
26	SI	Read/write	Source increment. When set the source address is incremented after each transfer.
25:24	Reserved	-	Reserved, read as zero, do not modify.
23:21	DWidth	Read/write	Destination transfer width. Transfers wider than the AHB master bus width are illegal. The source and destination widths can be different from each other. The hardware automatically packs and unpacks the data as required.
20:18	SWidth	Read/write	Source transfer width. Transfers wider than the AHB master bus width are illegal. The source and destination widths can be different from each other. The hardware automatically packs and unpacks the data as required.
17:15	DBSize	Read/write	Destination burst size. Indicates the number of transfers which make up a destination burst transfer request. This value must be set to the burst size of the destination peripheral, or if the destination is memory, to the memory boundary size. The burst size is the amount of data that is transferred when the DMACxBREQ signal goes active in the destination peripheral. The burst size is not related to the AHB HBURST signal.
14:12	SBSIZE	Read/write	Source burst size. Indicates the number of transfers which make up a source burst. This value must be set to the burst size of the source peripheral, or if the source is memory, to the memory boundary size. The burst size is the amount of data that is transferred when the DMACxBREQ signal goes active in the source peripheral. The burst size is not related to the AHB HBURST signal.

Table 3-19 DMACCxControl register (continued)

Bits	Name	Type	Function
11:0	TransferSize	Read/write	Transfer size. For writes, this field indicates the number of (Source width) transfers to perform when the PrimeCell SMDMAC is the flow controller. For reads, the transfer size indicates the number of transfers completed on the destination bus. Reading the register when the channel is active does not give useful information, as by the time that the software has processed the value read, the channel might have progressed. It is intended to be used only when a channel is enabled and then disabled. If the PrimeCell DMAC controller is not the flow controller the transfer size value is not used.

Table 3-20 shows the value of the DBSize or SBSsize bits and the corresponding burst sizes.

Table 3-20 Source or destination burst size

Bit value of DBSize or SBSsize	Source or destination burst transfer request size
0b000	1
0b001	4
0b010	8
0b011	16
0b100	32
0b101	64
0b110	128
0b111	256

Table 3-21 shows the value of the SWidth or DWidth bits and the corresponding width.

Table 3-21 Source or destination transfer width

Bit value of SWidth or DWidth	Source or destination width
0b000	Byte (8-bit)
0b001	Halfword (16-bit)
0b010	Word (32-bit)
0b011	Reserved
0b100	Reserved
0b101	Reserved
0b110	Reserved
0b111	Reserved

Protection and access information

AHB access information is provided to the source and destination peripherals when a transfer occurs. The transfer information is provided by programming the DMA channel (the Prot bit of the DMACCxControl register, and the Lock bit of the

DMACCxConfiguration register). These bits are programmed by software and peripherals can use this information if necessary. Three bits of information are provided, and Table 3-22 shows the purpose of the three protection bits.

Table 3-22 Protection bits

Bits	Description	Purpose
0	Privileged or User	Indicates that the access is in User, or privileged mode: 0 = User mode 1 = privileged mode. This bit controls the AHB HPROT[1] signal.
1	Bufferable or not bufferable	Indicates that the access is bufferable, or not bufferable: 0 = not bufferable 1 = bufferable. This bit indicates that the access is bufferable. This bit can, for example, be used to indicate to an AMBA bridge that the read can complete in zero wait states on the source bus without waiting for it to arbitrate for the destination bus and for the slave to accept the data. This bit controls the AHB HPROT[2] signal.
2	Cacheable or not cacheable	Indicates that the access is cacheable or not cacheable: 0 = not cacheable 1 = cacheable. This indicates that the access is cacheable. This bit can, for example, be used to indicate to an AMBA bridge that when it saw the first read of a burst of eight it can transfer the whole burst of eight reads on the destination bus, rather than pass the transactions through one at a time. This bit controls the AHB HPROT[3] signal.

3.4.20 Channel configuration registers, DMACCxConfiguration

The two DMACCxConfiguration registers are read/write and are used to configure the DMA channel. The registers are not updated when a new LLI is requested.

Table 3-23 shows the bit assignment of a DMACCxConfiguration register.

Table 3-23 DMACCxConfiguration register

Bits	Name	Type	Function
31:19	Reserved	-	Reserved, must be written as zero, masked on read.
18	H	Read/write	Halt: 0 = allow DMA requests 1 = ignore further source DMA requests. The contents of the channels FIFO are drained. This value can be used with the Active and Channel Enable bits to cleanly disable a DMA channel.
17	A	Read	Active: 0 = there is no data in the FIFO of the channel 1 = the FIFO of the channel has data. This value can be used with the Halt and Channel Enable bits to cleanly disable a DMA channel.
16	L	Read/write	Lock. When set this bit enables locked transfers.
15	ITC	Read/write	Terminal count interrupt mask. When cleared this bit masks out the terminal count interrupt of the relevant channel.
14	IE	Read/write	Interrupt error mask. When cleared this bit masks out the error interrupt of the relevant channel.
13:11	FlowCntrl	Read/write	Flow control and transfer type. This value is used to indicate the flow controller and transfer type. The flow controller can be the PrimeCell SMDMAC, the source peripheral, or the destination peripheral. The transfer type can be either memory-to-memory, memory-to-peripheral, peripheral-to-memory, or peripheral-to-peripheral.
10	Reserved	-	Reserved, must be written as zero, masked on read.
9:6	DestPeripheral	Read/write	Destination peripheral. This value selects the DMA destination request peripheral. This field is ignored if the destination of the transfer is to memory.

Table 3-23 DMACCxConfiguration register (continued)

Bits	Name	Type	Function
5	Reserved	-	Reserved, must be written as zero, masked on read.
4:1	SrcPeripheral	Read/write	Source peripheral. This value selects the DMA source request peripheral. This field is ignored if the source of the transfer is from memory.
0	E	Read/write	Channel enable. Reading this bit indicates whether a channel is currently enabled or disabled: 0 = channel disabled 1 = channel enabled. The Channel Enable bit status can also be found by reading the DMACEnbldChns register. A channel is enabled by setting this bit. A channel can be disabled by clearing the Enable bit. This causes the current AHB transfer (if one is in progress) to complete and the channel is then disabled. Any data in the channels FIFO is lost. Restarting the channel by simply setting the Channel Enable bit has unpredictable effects and the channel must be fully re-initialized. The channel is also disabled, and Channel Enable bit cleared, when the last LLI is reached or if a channel error is encountered. If a channel has to be disabled without losing data in a channels FIFO the Halt bit must be set so that further DMA requests are ignored. The Active bit must then be polled until it reaches 0, indicating that there is no data left in the channels FIFO. Finally the Channel Enable bit can be cleared.

Table 3-24 describes the bit values of the three flow control and transfer type bits.

Table 3-24 Flow control and transfer type bits

Bit value	Transfer type	Controller
000	Memory to memory	PrimeCell DMA
001	Memory to peripheral	PrimeCell DMA
010	Peripheral to memory	PrimeCell DMA

Table 3-24 Flow control and transfer type bits (continued)

Bit value	Transfer type	Controller
011	Source peripheral to destination peripheral	PrimeCell DMA
100	Source peripheral to destination peripheral	Destination peripheral
101	Memory to peripheral	Peripheral
110	Peripheral to memory	Peripheral
111	Source peripheral to destination peripheral	Source peripheral

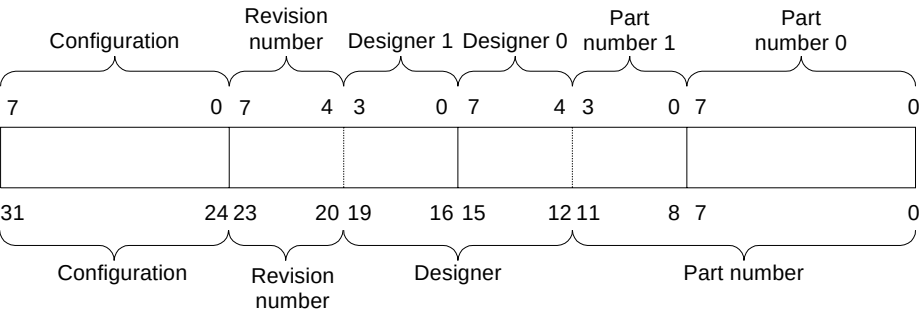
3.4.21 Peripheral identification registers, DMACPeriphID0-3

The DMACPeriphID0-3 registers are four 8-bit registers, that span address locations 0xFE0 - 0xFEC. The registers can conceptually be treated as a 32-bit register. The read only registers provide the following options of the peripheral:

- PartNumber[11:0]** This is used to identify the peripheral. The three digits product code 0x081 is used.
- Designer ID[19:12]** This is the identification of the designer. ARM Limited is 0x41 (ASCII A).
- Revision[23:20]** This is the revision number of the peripheral. The revision number starts from 0.
- Configuration[31:24]**
This is the configuration option of the peripheral.

Figure 3-1 on page 3-27 shows the bit assignment for the DMACPeriphID0-3 registers.

Actual register bit assignment



Conceptual register bit assignment

Figure 3-1 Peripheral identification register bit assignment

The four, 8-bit peripheral identification registers are described in the following subsections:

- *DMACPeriphID0 register on page 3-27*
- *DMACPeriphID1 register on page 3-28*
- *DMACPeriphID2 register on page 3-28*
- *DMACPeriphID3 register on page 3-29.*

DMACPeriphID0 register

The DMACPeriphID0 register is hard coded and the fields within the register determine the reset value. Table 3-25 shows the bit assignment of the DMACPeriphID0 register.

Table 3-25 DMACPeriphID0 register

Bits	Name	Description
31:8	-	Reserved, read undefined must be written as zeros
7:0	PartNumber0	These bits read back as 0x81

DMACPeriphID1 register

The DMACPeriphID1 register is hard coded and the fields within the register determine the reset value. Table 3-26 shows the bit assignment of the DMACPeriphID1 register.

Table 3-26 DMACPeriphID1 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must be written as zeros
7:4	Designer0	These bits read back as 0x1
3:0	PartNumber1	These bits read back as 0x0

DMACPeriphID2 register

The DMACPeriphID2 register is hard coded and the fields within the register determine the reset value. Table 3-27 shows the bit assignment of the DMACPeriphID2 register.

Table 3-27 DMACPeriphID2 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must be written as zeros
7:4	Revision	These bits read back as 0x0
3:0	Designer1	These bits read back as 0x4

DMACPeriphID3 register

The DMACPeriphID3 register is hard coded and the fields within the register determine the reset value. Table 3-28 shows the bit assignment of the DMACPeriphID3 register. The value of this register for this peripheral is 0x00.

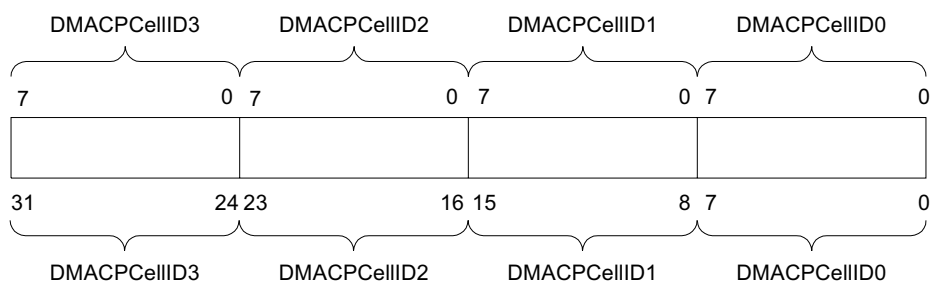
Table 3-28 DMACPeriphID3 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must be written as zeros.
7	Configuration	Indicates the number of DMA source requestors for the PrimeCell SMDMAC configuration: 0 = 16 DMA requestors 1 = 32 DMA requestors. This peripheral is set to 0.
6:4	Configuration	Indicates the AHB master bus width: 000 = 32-bit wide 001 = 64-bit wide 010 = 128-bit wide 011 = 256-bit wide 100 = 512-bit wide 101 = 1024-bit wide. This peripheral is set to 000.
3	Configuration	Indicates the number of AHB masters: 0 = one AHB master interface 1 = two AHB master interfaces. This peripheral is set to 0.
2:0	Configuration	Indicates the number of channels: 000 = 2 channels 001 = 4 channels 010 = 8 channels 011 = 16 channels 100 = 32 channels. This peripheral is set to 000.

3.4.22 PrimeCell identification registers, DMACPCellID0-3

The DMACPCellID0-3 registers are four 8-bit wide registers, that span address locations 0xFF0-0xFFC. The registers can conceptually be treated as a 32-bit register. The register is used as a standard cross-peripheral identification system. The DMACPCellID register is set to 0xB105F00D. Figure 3-2 shows the bit assignment for the DMACPCellID0-3 registers.

Actual register bit assignment



Conceptual register bit assignment

Figure 3-2 PrimeCell identification register bit assignment

The four, 8-bit PrimeCell identification registers are described in the following subsections:

- *DMACPCellID0 register* on page 3-31
- *DMACPCellID1 register* on page 3-31
- *DMACPCellID2 register* on page 3-31
- *DMACPCellID3 register* on page 3-32.

DMACPCellID0 register

The DMACPCellID0 register is hard coded and the fields within the register determine the reset value. Table 3-29 shows the bit assignment of the DMACPCellID0 register.

Table 3-29 DMACPCellID0 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must be written as zeros
7:0	DMACPCellID0	These bits read back as 0x00

DMACPCellID1 register

The DMACPCellID1 register is hard coded and the fields within the register determine the reset value. Table 3-30 shows the bit assignment of the DMACPCellID1 register.

Table 3-30 DMACPCellID1 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must be written as zeros
7:0	DMACPCellID1	These bits read back as 0xF0

DMACPCellID2 register

The DMACPCellID2 register is hard coded and the fields within the register determine the reset value. Table 3-31 shows the bit assignment of the DMACPCellID2 register.

Table 3-31 DMACPCellID2 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must be written as zeros
7:0	DMACPCellID2	These bits read back as 0x05

DMACPCellID3 register

The DMACPCellID3 register is hard coded and the fields within the register determine the reset value. Table 3-32 shows the bit assignment of the DMACPCellID3 register.

Table 3-32 DMACPCellID3 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must be written as zeros
7:0	DMACPCellID3	These bits read back as 0xB1

3.5 Address generation

Address generation can be either incrementing or non-incrementing (address wrapping is not supported). Bursts do not cross the 1KB address boundary.

3.6 Scatter/gather

Scatter/gather is supported through the use of linked lists. This means that the source and destination areas do not need to occupy contiguous areas in memory. Where scatter/gather is not required the DMACCxLLI register must be set to 0. For further details on scatter/gather DMA see Appendix C *Scatter/Gather*.

3.6.1 Linked list items

A LLI consists of four words. These words are organized in the following order:

1. DMACCxSrcAddr
2. DMACCxDestAddr
3. DMACCxLLI
4. DMACCxControl.

———— **Note** ————

The DMACCxConfiguration DMA channel configuration register is not part of the linked list item.

—————

3.6.2 Programming the PrimeCell SMDMAC for scatter/gather DMA

To program the Primecell SMDMAC for scatter/gather DMA:

1. Write the LLIs for the complete DMA transfer to memory. Each linked list item contains four words:
 - source address
 - destination address
 - pointer to next LLI
 - control word.

The last LLI has its linked list word pointer set to 0.

2. Choose a free DMA channel with the priority needed. DMA channel 0 has the highest priority and DMA channel 1 the lowest priority.
3. Write the first linked list item, previously written to memory, to the relevant channel in the PrimeCell SMDMAC.

4. Write the channel configuration information to the channel configuration register and set the Channel Enable bit. The PrimeCell SMDMAC then transfers the first and then subsequent packets of data as each linked list item is loaded.
5. An interrupt can be generated at the end of each LLI depending on the Terminal Count bit in the DMACCxControl register. If this bit is set an interrupt is generated at the end of the relevant LLI. The interrupt request must then be serviced and the relevant bit in the DMACIntTCClear register must be set to clear the interrupt.

If so this interrupt request must be serviced and the relevant IntTCClear bit in the DMACIntTCClr register set to clear the interrupt request interrupt.

3.7 Interrupt requests

Interrupt requests can be generated when an AHB error is encountered, or at the end of a transfer (terminal count) after all the data corresponding to the current LLI has been transferred to the destination. The interrupts can be masked by programming the relevant bits on the relevant DMACCxControl and DMACCxConfiguration channel registers. Interrupt status registers are provided which group the interrupt requests from all the DMA channels prior to interrupt masking (DMACRawIntTCStatus, DMACRawIntErrorStatus), and after interrupt masking (DMACIntTCStatus, DMACIntErrorStatus). The DMACIntStatus register combines both the DMACIntTCStatus and DMACIntErrorStatus requests into a single register to enable the source of an interrupt to be quickly found. Writing to the DMACIntTCClear or the DMACIntErrClr registers with a bit set HIGH allows selective clearing of interrupts.

The PrimeCell SMDMAC provides two interrupt request connection schemes, see *Interrupt controller connectivity* on page 2-10. The simplest connection scheme has a combined error and end of transfer complete interrupt request. To find the source of an interrupt both the DMACIntStatus and DMACIntTCStatus registers must be read.

For faster interrupt response an alternate connection scheme can be used. This scheme uses separate interrupt requests for the error and transfer complete requests. To find the source of an interrupt one of either the DMACIntTCStatus or DMACIntErrorStatus registers must be read.

3.7.1 Combined terminal count and error interrupt sequence flow

The following procedure must be followed when the DMACINTCOMBINE interrupt request is used:

1. Wait until the combined interrupt request from PrimeCell SMDMAC goes active.
2. Assuming the interrupt is enabled in the interrupt controller and in the processor, the processor branches to the interrupt vector address and enters the interrupt service routine.
3. Read the interrupt controllers status register and determine whether the source of the request was the PrimeCell SMDMAC.
4. Read the DMACIntStatus register to determine which channel generated the interrupt. If more than one request is active it is recommended that the highest priority channels must be checked first.
5. Read the DMACIntTCStatus register to determine whether the interrupt was generated due to the end of the transfer (terminal count) or due to an error occurring. A HIGH bit indicates that the transfer completed.

6. Read the DMACIntErrorStatus register to determine whether the interrupt was generated due to the end of the transfer (terminal count) or due to an error occurring. A HIGH bit indicates that an error occurred.
7. Write a 1 to the relevant bit in the DMACIntTCClear (or DMACIntErrClr) register to clear the interrupt request.

3.7.2 Terminal count interrupt sequence flow

When the separate, (DMACINTTC and DMACINTERROR), interrupt requests are used the following procedure must be followed:

1. Wait until the terminal count DMA interrupt request goes active.
2. Assuming the interrupt is enabled in the interrupt controller and in the processor, the processor branches to the interrupt vector address and enters the interrupt service routine.
3. Read the interrupt controllers status register to determine the source of the interrupt request was the PrimeCell SMDMAC asserting the **DMACINTTC** signal.
4. Read the DMACIntTCStatus register to determine which channel generated the interrupt. If more than one request is active it is recommended that the highest priority channel must be serviced first.
5. Service the interrupt request.
6. Write a 1 to the relevant bit in the DMACIntTCClear register to clear the interrupt request.

3.7.3 Error interrupt sequence flow

When the separate, (DMACINTTC and DMACINTERR), interrupt requests are used the following procedure must be followed:

1. Wait until the interrupt request due to a DMA channel error goes active.
2. Assuming the interrupt is enabled in the interrupt controller and in the processor, the processor branches to the interrupt vector address and enters the interrupt service routine.
3. Read the interrupt controllers status register and determine the source of the request was the PrimeCell SMDMAC asserting the **DMACINTERR** signal.

4. Read the DMACIntErrorStatus register to determine which channel generated the interrupt. If more than one request is active it is recommended that the highest priority channels are checked first.
5. Service the interrupt request.
6. Write a 1 to the relevant bit in the DMACIntErrClr register to clear the interrupt request.

3.7.4 Interrupt polling sequence flow

The PrimeCell SMDMAC interrupt request signal is either masked out, disabled in the interrupt controller or disabled in the processor. The following procedure must be followed when polling the PrimeCell SMDMAC:

1. Read the DMACIntStatus register. If none of the bits are HIGH repeat this step, otherwise go to step 2. If more than one request is active it is recommended that the highest priority channels must be checked first.
2. Read the DMACIntTCStatus register to determine whether the interrupt was generated due to the end of the transfer (terminal count) or due to an error occurring. A HIGH bit indicates that the transfer completed.
3. Service the interrupt request.
4. For an error interrupt write a 1 to the relevant bit of the DMACIntErrClr register to clear the interrupt request. For a terminal count interrupt write a 1 to the relevant bit of the DMACIntTCClr register.

3.8 PrimeCell SMDMAC data flow

This section describes the PrimeCell SMDMAC data flow sequences for:

- *Peripheral-to-memory, or memory-to-peripheral DMA flow* on page 3-39
- *Peripheral-to-peripheral DMA flow* on page 3-39
- *Memory-to-memory DMA flow* on page 3-41.

3.8.1 Peripheral-to-memory, or memory-to-peripheral DMA flow

For a peripheral-to-memory, or memory-to-peripheral DMA flow the following sequence occurs:

1. Program and enable the DMA channel.
2. Wait for a DMA request.
3. When:
 - a. The DMA request goes active.
 - b. The DMA stream has the highest pending priority.
 - c. The PrimeCell SMDMAC is the bus master of the AHB bus.
 The PrimeCell SMDMAC then starts transferring data.
4. If an error occurs while transferring the data an error interrupt is generated and disables the DMA stream, go to step 7.
5. Decrement the transfer count if the PrimeCell SMDMAC is controlling the flow control.
6. If the transfer has completed (indicated by the transfer count reaching 0 if the PrimeCell SMDMAC is performing flow control, or by the peripheral setting the **DMACLBREQ** or **DMACLSREQ** signals if the peripheral is performing flow control) the following occurs:
 - a. The PrimeCell SMDMAC asserts the **DMACTC** signal.
 - b. The terminal count interrupt is generated (this interrupt can be masked).
 - c. If the **DMACCxLLI** register is not 0, then reload the **DMACCxSrcAddr**, **DMACCxDestAddr**, **DMACCxLLI** and **DMACCxControl** registers and go to back to step 2. However, if **DMACCxLLI** is 0, go to step 7.

3.8.2 Peripheral-to-peripheral DMA flow

For a peripheral-to-peripheral DMA flow the following sequence occurs:

1. Program and enable the DMA channel.

2. Wait for a source DMA request.
3. When:
 - a. The DMA request goes active.
 - b. The DMA stream has the highest pending priority.
 - c. The PrimeCell SMDMAC is the bus master of the AHB bus.The PrimeCell SMDMAC then starts transferring data.
4. If an error occurs while transferring the data an error interrupt is generated, then finish.
5. Decrement the transfer count if the PrimeCell SMDMAC is controlling the flow control.
6. If the transfer has completed (indicated by the transfer count reaching 0 if the PrimeCell SMDMAC is performing flow control, or by the peripheral setting the **DMACLBREQ** or **DMACLSREQ** signals if the peripheral is performing flow control) the following occurs:
 - a. The PrimeCell SMDMAC asserts the **DMACTC** signal to the source peripheral.
 - b. Further source DMA requests are ignored.
7. When the destination DMA request goes active and there is data in the PrimeCell SMDMAC FIFO, transfer data into the destination peripheral.
8. If an error occurs while transferring the data, generate an error interrupt, go to step 10.
9. If the transfer has completed it is indicated by the transfer count reaching 0 if the PrimeCell SMDMAC is performing flow control, or by the peripheral setting the **DMACLBREQ** or **DMACLSREQ** signals if the peripheral is performing flow control. The following happens:
 - a. The PrimeCell SMDMAC asserts the **DMACTC** signal to the destination peripheral.
 - b. The terminal count interrupt is generated (this interrupt can be masked).
 - c. If the DMACCxLLI register is not 0, then reload the DMACCxSrcAddr, DMACCxDestAddr, DMACCxLLI, and DMACCxControl registers and go to back to step 2. However, if DMACCxLLI is 0, go to step 10.

3.8.3 Memory-to-memory DMA flow

For a memory-to-memory DMA flow the following sequence occurs:

1. Program and enable the DMA channel.
2. Transfer data whenever the DMA channel has the highest pending priority and the PrimeCell SMDMAC gains bus master ship of the AHB bus.
3. If an error occurs while transferring the data generate an error interrupt and disable the DMA stream.
4. Decrement the transfer count.
5. If the count has reached zero:
 - a. Generate a terminal count interrupt (the interrupt can be masked).
 - b. If the DMACCxLLI register is not 0, then reload the DMACCxSrcAddr, DMACCxDestAddr, DMACCxLLI, and DMACCxControl registers and go to back to step 2. However, if DMACCxLLI is 0, disable the DMA channel.

Chapter 4

Programmer's Model for Test

This chapter describes the additional logic for integration testing. It contains the following sections:

- *PrimeCell SMDMAC test harness overview* on page 4-2
- *Scan testing* on page 4-3
- *Test registers* on page 4-4
- *Integration test* on page 4-7.

4.1 PrimeCell SMDMAC test harness overview

The additional logic for functional verification and integration vectors allows:

- capture of input signals to the block
- stimulation of the output signals.

The integration vectors provide a way of verifying that the PrimeCell SMDMAC is correctly wired into a system. This is done by separately testing two groups of signals:

AMBA signals

These are tested by checking the connections of all the address data bits.

Intra-chip signals (such as interrupt sources)

The tests for these signals are system-specific, and enable you to write the necessary tests. Additional logic is implemented allowing you to read and write to each intra-chip input/output signal.

These test features are controlled by test registers. This allows you to test the PrimeCell SMDMAC in isolation from the rest of the system using only transfers from the AMBA AHB.

4.2 Scan testing

The PrimeCell SMDMAC has been designed to simplify:

- insertion of scan test cells
- use of *Automatic Test Pattern Generation* (ATPG).

This the recommended method of manufacturing test.

4.3 Test registers

The PrimeCell SMDMAC test registers are memory-mapped as shown in Table 4-1.

Table 4-1 Test registers memory map

Address	Type	Width	Reset value	Name	Description
SMDMAC base +0x500	Read/write	1	0x0	DMACITCR	This register is used to select the various test modes. This register is cleared upon reset.
SMDMAC base +0x504	Read/write	16	0x0000	DMACITOP1	Control and read the DMACxCLR[15:0] output lines.
SMDMAC base +0x508	Read/write	16	0x0000	DMACITOP2	Control and read the DMACxTC[15:0] output lines.
SMDMAC base +0x50C	Read/write	2	0x0	DMACITOP3	Control and read the DMACINTERR and DMACINTTC output lines.

Each register shown in Table 4-1 is described in the following sections:

- *Test control register, DMACITCR* on page 4-4
- *Integration test output register 1, DMACITOP1* on page 4-5
- *Integration test output register 2, DMACITOP2* on page 4-5
- *Integration test output register 3, DMACITOP3* on page 4-6.

4.3.1 Test control register, DMACITCR

DMACITCR register is read/write, it is used to select the various test modes and is cleared on reset. The register allows the PrimeCell SMDMAC to be tested using TIC block level tests and BIST integration and system level tests.

Table 4-2 shows the bit assignment of the DMACITCR.

Table 4-2 DMACITCR register bits

Bits	Name	Description
15:1	-	Reserved, unpredictable when read.
0	T	Test mode enable. Multiplex the test registers to control the input and output lines: 0 = normal operation 1 = test registers multiplexed onto input and outputs.

4.3.2 Integration test output register 1, DMACITOP1

DMACITOP1 is a 16-bit read/write register. It is used for controlling and reading the **DMACCLR[15:0]** output lines in test mode.

Table 4-3 shows the bit assignment of the DMACITOP1.

Table 4-3 DMACITOP1 register bits

Bits	Name	Description
15:0	DMACCLR	The DMACCLR[15:0] response outputs can be set to a certain value in test mode by writing to the register. A read returns the value on the outputs (after the test multiplexor).

4.3.3 Integration test output register 2, DMACITOP2

DMACITOP2 is a 16-bit read/write register. It is used for controlling and reading the **DMACTC[15:0]** output lines in test mode.

Table 4-4 shows the bit assignment of the DMACITOP2.

Table 4-4 DMACITOP2 register bits

Bits	Name	Description
15:0	DMACTC	The DMACTC[15:0] response outputs can be set to a certain value in test mode by writing to the register. A read returns the value on the outputs (after the test multiplexor).

4.3.4 Integration test output register 3, DMACITOP3

DMACITOP3 is a 16-bit read/write register. It is used to control and read the interrupt request output lines in test mode.

Table 4-5 shows the bit assignments for the DMACITOP3.

Table 4-5 DMACITOP3 register bits

Bits	Name	Description
15:2	-	Reserved, unpredictable when read.
1	TC	The DMACINTTC interrupt request can be set to a certain value in test mode by writing to the register. A read returns the value on the output (after the test multiplexor).
0	E	The DMACINTERROR interrupt request can be set to a certain value in test mode by writing to the register. A read returns the value on the output (after the test multiplexor).

———— **Note** ————

The **DMACINTCOMBINE** interrupt request signal combines both interrupt requests, (**DMACINTTC** and **DMACINTERROR**), into one interrupt request signal. Therefore if or both of the TC and E bits are set then **DMACINTCOMBINE** is active.

4.4 Integration test

The non-AMBA intra-chip input signals can be set to certain values and the output signals can be read using test registers.

You can use the test control register (DMACITCR) to set the test multiplexors into test mode.

4.4.1 Input signals

The input signals can be set as follows:

- **DMACxBREQ[15:0]** can be set using the DMACSoftBReq register. The status of the DMACxBREQ inputs can be read after being combined with SoftBReq by reading the DMACSoftBReq register.
- **DMACxSREQ[15:0]** can be set using the DMACSoftSReq register. The status of the DMACxSREQ inputs can be read after being combined with SoftSReq by reading the DMACSoftSReq register.
- **DMACxLBREQ[15:0]** can be set using the DMACSoftLBReq register. The status of the DMACxLBREQ inputs can be read after being combined with SoftLBReq by reading the DMACSoftLBReq register.
- **DMACxLSREQ[15:0]** can be set using the DMACSoftLSReq register. The status of the DMACxLSREQ inputs can be read after being combined with SoftLSReq by reading the DMACSoftLSReq register.

4.4.2 Output signals

The output signals can be set as follows:

- **DMACxCLR[15:0]** can be set by writing to the DMACITOP1 register. A read returns the value on the outputs (after the test multiplexor).
- **DMACxTC[15:0]** can be set by writing to the DMACITOP2 register. A read returns the value on the outputs (after the test multiplexor).
- **DMACINTERROR** can be set by writing to the DMACITOP3 register. A read returns the value on the outputs (after the test multiplexor).
- **DMACINTTC** can be set by writing to the DMACITOP3 register. A read returns the value on the outputs (after the test multiplexor).

Appendix A

ARM PrimeCell SMDMAC (PL081) Signal Descriptions

This appendix describes the signals that interface with the ARM PrimeCell Single Master DMA Controller (PL081) block. It contains the following sections:

- *DMA interrupt request signals* on page A-2
- *DMA request and response signals* on page A-3
- *AHB slave signals* on page A-4
- *AHB master signals* on page A-6
- *AHB master bus request signals* on page A-8.

A.1 DMA interrupt request signals

Table A-1 describes the DMA interrupt request signals.

Table A-1 DMA interrupt request signal descriptions			
Name	Type	Destination	Description
DMACINTERROR	Output	Interrupt controller	DMA error interrupt request to interrupt controller.
DMACINTTC	Output	Interrupt controller	DMA count interrupt request to interrupt controller.
DMACINTCOMBINE	Output	Interrupt controller	DMA request to interrupt controller. This signal combines the DMACINTERROR and DMACINTTC requests.

A.2 DMA request and response signals

Table A-2 describes the DMA request and response signals.

Table A-2 DMA request and response signal descriptions

Name	Type	Source/ destination	Description
DMACBREQ[15:0]	Input	DMA peripheral	DMA burst transfer request.
DMACSREQ[15:0]	Input	DMA peripheral	DMA single transfer request.
DMACLBREQ[15:0]	Input	DMA peripheral	DMA last burst transfer request.
DMACLSREQ[15:0]	Input	DMA peripheral	DMA last single transfer request.
DMACLR[15:0]	Output	DMA peripheral	DMA request acknowledge clear.
DMACTC[15:0]	Output	DMA peripheral	DMA terminal count. Indicates that the transaction is complete and the packet of data is transferred.

A.3 AHB slave signals

Table A-3 describes the AHB slave signals.

Table A-3 AHB slave signal descriptions

Name	Type	Source/ destination	Description
HCLK	Input	Clock generator	This clock times all bus transfers. All signal timings are related to the rising edge of HCLK .
HRESETn	Input	Reset controller	The bus reset signal is active LOW and is used to reset the system and the bus. This is the only active LOW signal.
HADDR[11:2]	Input	AHB master	The system address bus.
HWRITE	Input	AHB master	Transfer direction. When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZE[2:0]	Input	AHB master	Indicates the size of the transfer, all transfers to and from the PrimeCell Single Master DMA Controller must be 32-bit (HSIZE[2:0] = 0b010).
HTRANS	Input	AHB master	Transfer type, which can be IDLE, BUSY, NONSEQUENTIAL, or SEQUENTIAL. This signal must be connected to HTRANS[1] on the AHB interface. HTRANS[0] is not used.
HWDATA[31:0]	Input	AHB master	The write data bus is used to transfer data from the master to the bus slaves during write operations. A minimum data bus width of 32 bits is recommended. However, this can easily be extended to allow for higher bandwidth operation.
HSELDMAC	Input	Decoder	The PrimeCell Single Master DMA Controller AHB slave has its own slave select signal. This signal indicates that the current transfer is intended for the selected slave. This signal is simply a combinatorial decode of the address bus.

Table A-3 AHB slave signal descriptions (continued)

Name	Type	Source/ destination	Description
HRDATA[31:0]	Output	AHB master	The read data bus is used to transfer data from bus slaves to the bus master during read operations. A minimum data bus width of 32 bits is recommended. However, this can easily be extended to allow for higher bandwidth operation.
HREADYIN	Input	External slave	When HIGH, the HREADYIN signal indicates that a transfer has finished on the bus. This signal can be driven LOW to extend a transfer.
HREADYOUT	Output	AHB master	When HIGH, the HREADYOUT signal indicates that a transfer has finished on the bus. This signal can be driven LOW to extend a transfer.
HRESP[1:0]	Output	AHB master	The transfer response provides additional information on the status of a transfer. Four different responses are provided, OKAY, ERROR, RETRY, and SPLIT. The PrimeCell Single Master DMA Controller only generates the OKAY and ERROR responses.

A.4 AHB master signals

Table A-4 describes the AHB master signals.

Table A-4 AHB master signal descriptions

Name	Type	Source/ destination	Description
HADDRM[31:0]	Output	AHB slave	32-bit system address bus.
HTRANSM[1:0]	Output	AHB slave	Indicates the type of the current transfer, which can be NONSEQUENTIAL, SEQUENTIAL, IDLE, or BUSY.
HWRITEM	Output	AHB slave	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZEM[2:0]	Output	AHB slave	Indicates the size of the transfer, which is typically byte (8-bit), halfword (16-bit), or word (32-bit). The PrimeCell Single Master DMA Controller allows 8, 16, and 32-bit transfer widths: 8-bit: HSIZE[2:0] = 0b000 16-bit: HSIZE[2:0] = 0b001 32-bit: HSIZE[2:0] = 0b010.
HPROTM[3:0]	Output	AHB slave	Protection control. Provides information about a bus access.
HLOCKDMACM	Output	AHB slave	This signal is used to indicate to the arbiter that the requesting master is going to perform a number of indivisible transfers. It also indicates the arbiter must not grant the bus to another bus master once the first of the locked transfers has commenced.
HBURSTM[2:0]	Output	AHB slave	Indicates if the transfer is a burst transfer.
HWDATAM[31:0]	Output	AHB slave	The write data bus is used to transfer data from the master to the bus slaves during write operations.

Table A-4 AHB master signal descriptions (continued)

Name	Type	Source/ destination	Description
HRDATAM[31:0]	Input	AHB slave	The read data bus is used to transfer data from bus slaves to the bus master during read operations.
HREADYINM	Input	AHB slave	When HIGH, the HREADY signal indicates that a transfer has finished on the bus. This signal can be driven LOW to extend a transfer.
HRESPM[1:0]	Input	AHB slave	The transfer response provides additional information on the status of a transfer. Four different responses are provided, OKAY, ERROR, RETRY and SPLIT. The OKAY response is used by the slave to indicate that the transfer has completed successfully. The ERROR response is used by the slave to indicate that an error has occurred. The Single Master DMA Controller then asserts the Error Interrupt request. The RETRY and SPLIT responses are used by the slave to indicate that the data isn't ready. The Single Master DMA Controller must retry the transfer later.

A.5 AHB master bus request signals

Table A-5 describes the AHB master bus request signals.

Table A-5 AHB master bus request signal descriptions			
Name	Type	Source/ destination	Description
HBUSREQDMACM	Output	Arbiter	Bus request signal used by the PrimeCell Single Master DMA Controller to request the AHB bus.
HGRANTDMACM	Input	Arbiter	This signal is used to indicate that the DMA master is selected. The master gains bus ownership when HGRANTDMAC and HREADY are HIGH on the rising edge of HCLK .

A.6 Scan test control signals

Table A-6 describes the internal scan test control signals.

Table A-6 Internal scan test control signal descriptions

Name	Type	Source/ destination	Description
SCANENABLE	Input	Scan controller	Scan enable
SCANINHCLK	Input	Scan controller	Scan data input for HCLK domain
SCANOUTHCLK	Output	Scan controller	Scan data output for HCLK domain

Appendix B

DMA Interface

This section describes the DMA request and response interface. It contains the following sections:

- *DMA request signals* on page B-2
- *DMA response signals* on page B-3
- *Flow control* on page B-4
- *Transfer types* on page B-5
- *Signal timing* on page B-21
- *Functional timing diagram* on page B-22
- *PrimeCell SMDMAC transfer timing diagram* on page B-23.

B.1 DMA request signals

The DMA request signals are used by peripherals to request a data transfer. The DMA request signals indicate:

- whether a single word or a burst (multi-word) transfer of data is required
- whether the transfer is the last in the data packet.

The DMA request signals to the PrimeCell SMDMAC for each peripheral are:

DMACxBREQ Burst request signal. This causes a programmed burst number of words to be transferred.

DMACxSREQx Single transfer request signal. This causes a single word to be transferred. The PrimeCell SMDMAC transfers a single word to, or from the peripheral.

DMACxLBREQx Last burst request signal.

DMACxLSREQx Last single transfer request signal.

———— **Note** ————

If a peripheral transfers only bursts of data, it is not necessary to connect the single transfer request signal. If a peripheral transfers only single words of data, it is not necessary to connect the burst request signal.

B.2 DMA response signals

The DMA response signals indicate whether the transfer initiated by the DMA request signal has completed. The response signals can also be used to indicate whether a complete packet has been transferred.

The DMA response signals from the PrimeCell SMDMAC for each peripheral are:

DMACxCLR_x DMA clear or acknowledge signal.

DMACxTC DMA terminal count signal.

The **DMACxCLR_x** signal is used by the PrimeCell SMDMAC to acknowledge a DMA request from the peripheral.

The **DMACxTC** signal is used by the PrimeCell SMDMAC to indicate to the peripheral that the DMA transfer is complete.

———— **Note** —————

Some peripherals do not require connection to the DMA terminal count signal.

B.3 Flow control

The peripheral that controls the length of the packet is known as the *flow controller*. The flow controller is usually the PrimeCell SMDMAC where the packet length is programmed by software before the DMA channel is enabled. If the packet length is unknown when the DMA channel is enabled, either the source or destination peripherals can be used as the flow controller.

For simple or low performance peripherals that know the packet length (that is, when the peripheral is the flow controller), a simple way to indicate that a transaction has completed is for the peripheral to generate an interrupt and allow the processor to reprogram the DMA channel.

For higher performance peripherals where the peripheral is the flow controller, use the PrimeCell SMDMAC flow control signals:

DMACLBREQ DMA last burst request.

DMACLSREQ DMA last single request.

The transfer size value (in the DMACCxControl register) is ignored if a peripheral is configured as the flow controller.

When the DMA is transferred:

1. The **DMACTC** signal goes active to indicate that the transfer has finished.
2. A TC interrupt is generated, if enabled.
3. The PrimeCell SMDMAC moves on to the next LLI.

B.4 Transfer types

The PrimeCell SMDMAC allows for four transfer types:

- memory-to-peripheral
- peripheral-to-memory
- memory-to-memory
- peripheral-to-peripheral.

Each transfer type can have either the peripheral or the PrimeCell SMDMAC as the flow controller so there are eight possible control scenarios.

Table B-1 indicates the request signals used for each type of transfer.

Table B-1 DMA request signal usage

Transfer direction	Request generator	Flow controller	Request signals used
Memory-to-peripheral	Peripheral	PrimeCell SMDMAC	DMACBREQ
Memory-to-peripheral	Peripheral	Peripheral	DMACBREQ, DMACSREQ, DMACLBREQ, DMACLSREQ
Peripheral-to-memory	Peripheral	PrimeCell SMDMAC	DMACBREQ, DMACSREQ
Peripheral-to-memory	Peripheral	Peripheral	DMACBREQ, DMACSREQ, DMACLBREQ, DMACLSREQ
Memory-to-memory	PrimeCell SMDMAC	PrimeCell SMDMAC	None
Source peripheral to destination peripheral	Source peripheral and destination peripheral	Source peripheral	Source, DMACBREQ, DMACSREQ, DMACLBREQ, DMACLSREQ Destination, DMACBREQ
Source peripheral to destination peripheral	Source peripheral and destination peripheral	Destination peripheral	Source, DMACBREQ, DMACSREQ Destination, DMACBREQ, DMACSREQ, DMACLBREQ, DMACLSREQ
Source peripheral to destination peripheral	Source peripheral and destination peripheral	PrimeCell SMDMAC	Source, DMACBREQ, DMACSREQ Destination, DMACBREQ

B.4.1 Peripheral-to-memory transaction under PrimeCell SMDMAC flow control

For transactions comprising bursts, use the burst request signal **DMACBREQ**, as shown in Figure B-1 on page B-6.

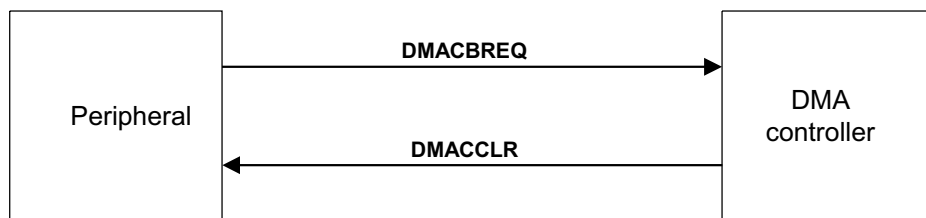


Figure B-1 Peripheral-to-memory transaction comprising bursts

For transactions comprising single requests, use the single request signal **DMACSREQ** as shown in Figure B-2.

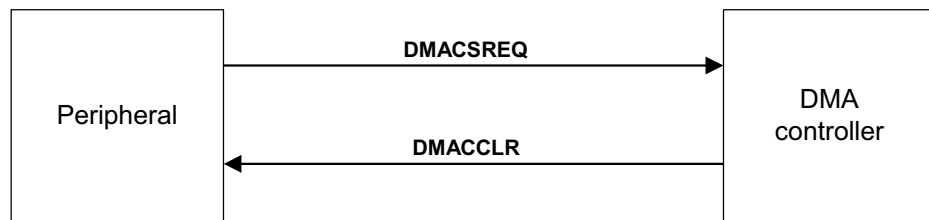


Figure B-2 Peripheral-to-memory transaction comprising single requests

For transactions that are not a multiple of the burst size, use both the burst and single request signals as shown in Figure B-3.

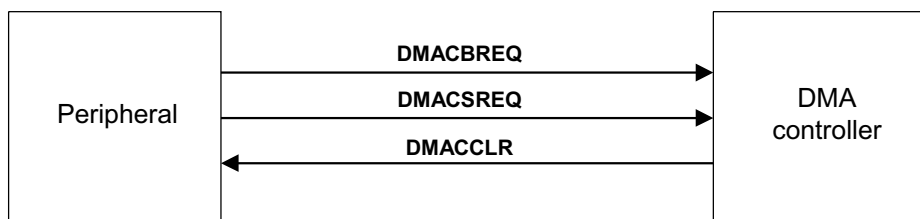


Figure B-3 Peripheral-to-memory transaction comprising bursts and single requests

The two request signals are not mutually exclusive. The PrimeCell SMDMAC monitors **DMACBREQ**, while the amount of data left to transfer is greater than the burst size, and commences a burst transfer (from the peripheral) when requested to do so. When the amount of data left is less than the burst size, the PrimeCell SMDMAC monitors **DMACSREQ** and commences single transfers when requested.

B.4.2 Memory-to-memory transaction under PrimeCell SMDMAC flow control

Software programs a DMA channel memory-to-memory transfer. Once enabled, the DMA channel commences transfers without DMA requests. It continues until one of the following occurs:

- all the data is transferred
- the channel is disabled by software.

Note

You must program memory-to-memory transfers with a low channel priority, otherwise the other DMA channels cannot access the bus until the memory-to-memory transfer has finished, or other AHB masters cannot perform any transaction.

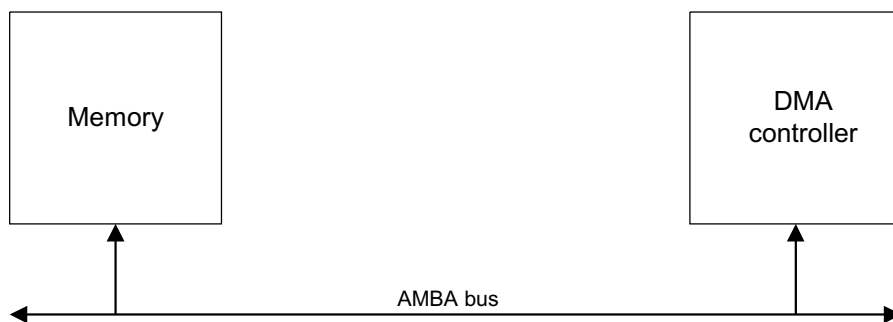


Figure B-4 Memory-to-memory transaction under DMA flow control

B.4.3 Peripheral-to-peripheral transfer under PrimeCell SMDMAC flow control

For transactions that are a multiple of the burst size, use the burst request signal **DMACBREQ**, as shown in Figure B-5.

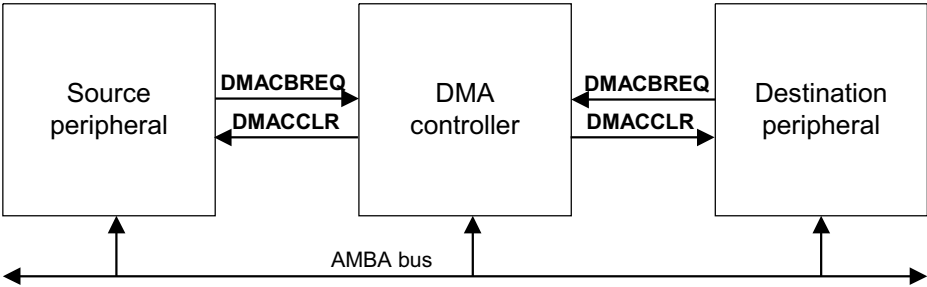


Figure B-5 Peripheral-to-peripheral transaction comprising bursts

For transactions comprising single transfers, use the single request signal **DMACSREQ**, as shown in Figure B-6.

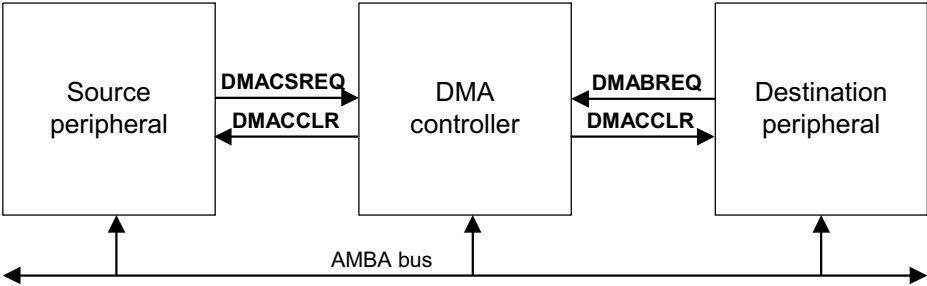


Figure B-6 Peripheral-to-peripheral transaction comprising single transfers

When the transaction is not a multiple of the burst size, use the following signals:

- the single and burst request signals (**DMACBREQ** and **DMACSREQ**) of the source peripheral
- the burst request signal (**DMACBREQ**) of the destination peripheral.

This is shown in Figure B-7.

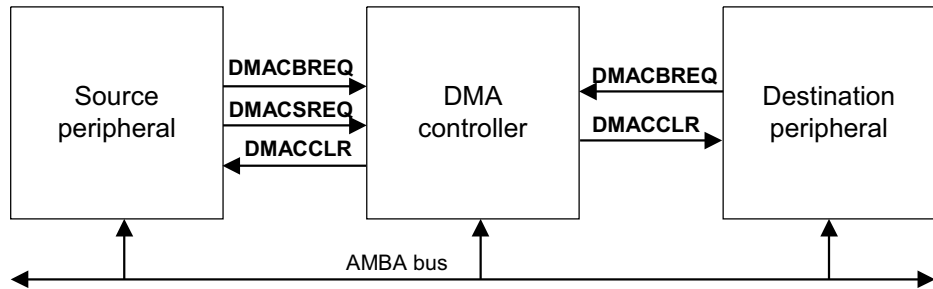


Figure B-7 Peripheral-to-peripheral transaction comprising bursts and single requests

The source peripheral follows the same procedure as that described in *Peripheral-to-memory transaction under PrimeCell SMDMAC flow control* on page B-5. The destination peripheral follows the same procedure as that described in *Memory-to-peripheral transaction under peripheral flow control* on page B-11. The next LLI is loaded when all read and write transfers are complete. You can use the **DMACTC** signal to indicate to the peripherals when the last data has been transferred.

B.4.4 Memory-to-peripheral transaction under peripheral flow control

For transactions that are a multiple of the burst size, use the burst request and last burst request signals, **DMACBREQ** and **DMACLBREQ**, as shown in Figure B-8.

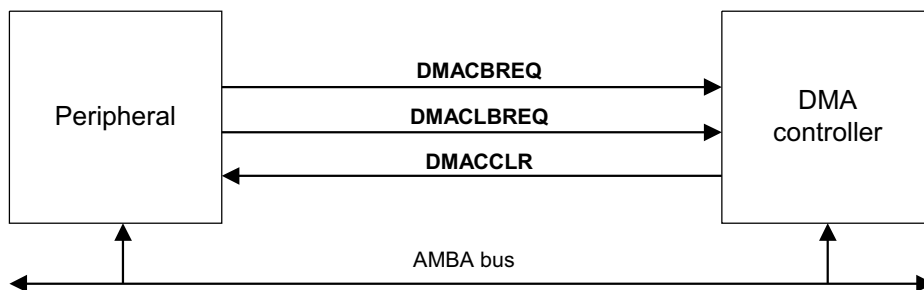


Figure B-8 Memory-to-peripheral transaction comprising bursts

The **DMACBREQ** and **DMACLBREQ** signals are mutually exclusive. The **DMACLBREQ** signal must be asserted to perform the last burst transfer.

For transactions comprising single transfers, use the single request signal and last single request signals, **DMACSREQ** and **DMACLSREQ**, as shown in Figure B-9.

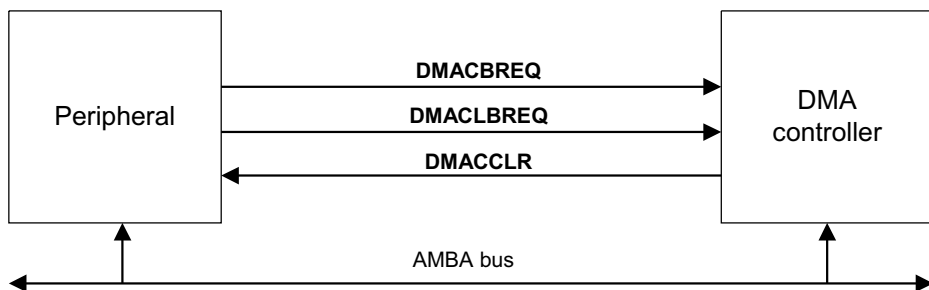


Figure B-9 Memory-to-peripheral transaction comprising single transfers

The **DMACSREQ** and **DMACLSREQ** signals are mutually exclusive. The **DMACLSREQ** signal must be asserted to perform the last single transfer.

For transactions that use burst transfers and where the transaction is not a multiple of the burst size, use the single and burst request signals, **DMACBREQ**, **DMACLBREQ**, **DMACSREQ**, and **DMACLSREQ**, as shown in Figure B-10.

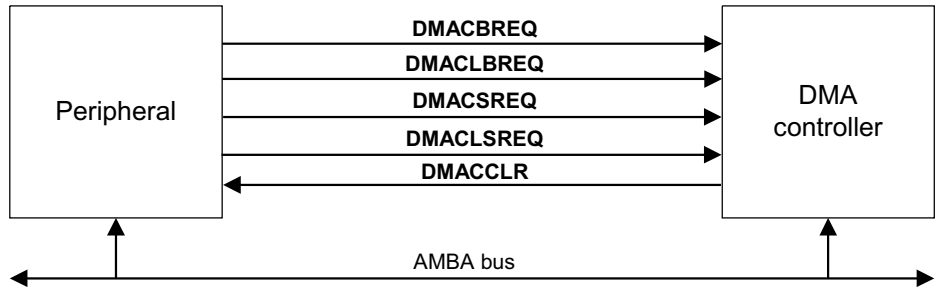


Figure B-10 Memory-to-peripheral transaction comprising bursts and single transfers

The four request signals in Figure B-10 are created mutually exclusive to each other. Each is asserted only when required. This means, for example, that a **DMACSREQ** is not replaced by a **DMACBREQ** when more data arrives if the **DMACSREQ** has not been serviced in time. The PrimeCell SMDMAC has no knowledge of the total length of transfer and initiates burst or single transfers to and from the peripheral as requested. The peripheral asserts burst requests using **DMACBREQ** until the amount of data still to be transferred is less than or equal to the burst size. At this point, if the remaining data is equal to the burst size then a burst request is issued using **DMACLBREQ**. Otherwise, single requests are issued on **DMACSREQ** until the last data item is ready, when **DMACLSREQ** is used.

When a last request (**DMACLBREQ** or **DMACLSREQ**) is made, the PrimeCell SMDMAC initiates the appropriate transfer then moves onto the next LLI.

B.4.5 Peripheral-to-memory transactions under peripheral flow control

For transactions comprising bursts, use the burst request and last burst request signals, (**DMACBREQ** and **DMACLBREQ**), as shown in Figure B-11.

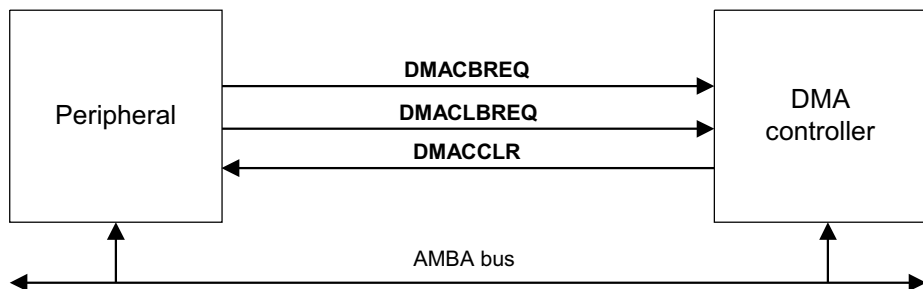


Figure B-11 Peripheral-to-memory transaction comprising bursts

The **DMACBREQ** and **DMACLBREQ** signals are mutually exclusive. The **DMACLBREQ** signal must be asserted to perform the last burst transfer.

For transactions comprising single transfers, use the single request and last single request signals, **DMACSREQ** and **DMACLSREQ**, as shown in Figure B-12.

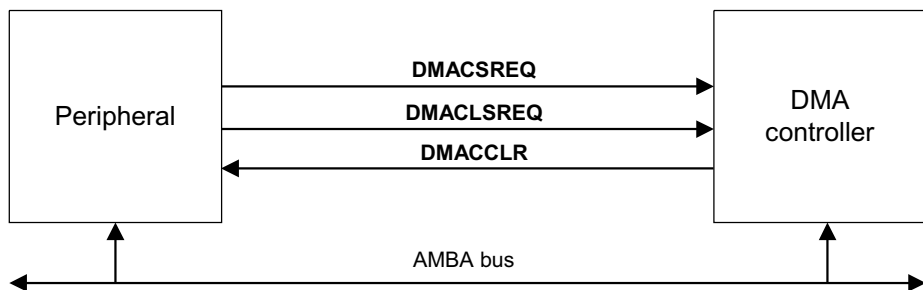


Figure B-12 Peripheral-to-memory transaction comprising single transfers

The **DMACSREQ** and **DMACLSREQ** signals are mutually exclusive. The **DMACLSREQ** signal must be asserted to perform the last single transfer.

For transactions that use burst transfers and where the transaction is not a multiple of the burst size, use the single and burst request signals, **DMACBREQ**, **DMACLBREQ**, **DMACSREQ** and **DMACLSREQ**, as shown in Figure B-13.

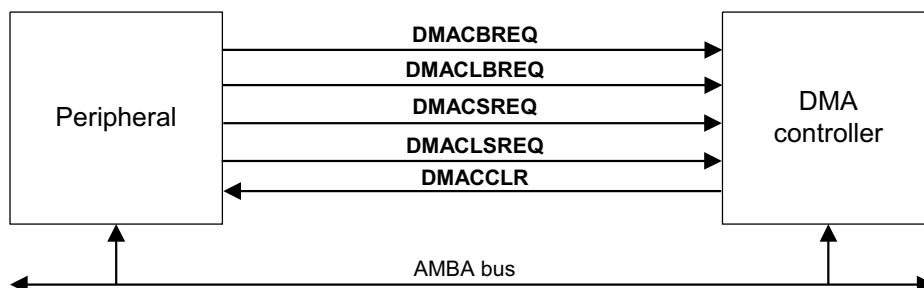


Figure B-13 Peripheral-to-memory transaction comprising bursts and single transfers

The four request signals are created mutually exclusive to each other. Each is only asserted when required. This means, for example, that a **DMACSREQ** is not replaced by a **DMACBREQ** when more data arrives if the **DMACSREQ** has not been serviced in time. The PrimeCell SMDMAC has no knowledge of the total length of transfer and initiates bursts or single transfers to and from the peripheral as requested. The peripheral asserts burst requests using **DMACBREQ** until the amount of data still to be transferred is less than or equal to the burst size. At this point, if the remaining data is equal to the burst size, a burst request using **DMACLBREQ** is issued. Otherwise, single requests are issued on **DMACSREQ** until the last data item is ready, when **DMACLSREQ** is used. Once a last request (**DMACLBREQ** or **DMACLSREQ**) is made, the PrimeCell SMDMAC initiates the appropriate transfer then moves onto the next LLI.

B.4.6 Peripheral-to-peripheral transactions under source peripheral flow control

For transactions that are a multiple of the burst size, use the following signals:

- the source burst request and last burst request signals **DMACBREQ** and **DMACLBREQ**
- the destination burst request signal **DMACBREQ**.

This is shown in Figure B-14.

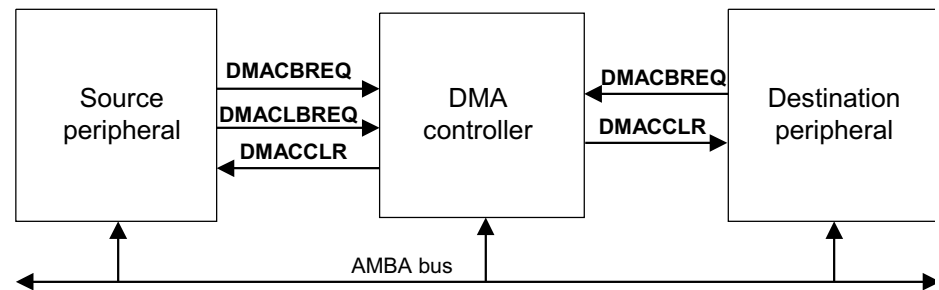


Figure B-14 Peripheral-to-peripheral transaction comprising bursts

The source signals **DMACBREQ** and **DMACLBREQ** signals are mutually exclusive. The **DMACLBREQ** signal must be asserted to perform the last burst transfer.

For transactions comprising single transfers, use the following signals:

- the source single request signal and last single request signal, **DMACSREQ** and **DMACLSREQ**
- the destination burst request signal, **DMACBREQ**.

This is shown in Figure B-15 on page B-16.

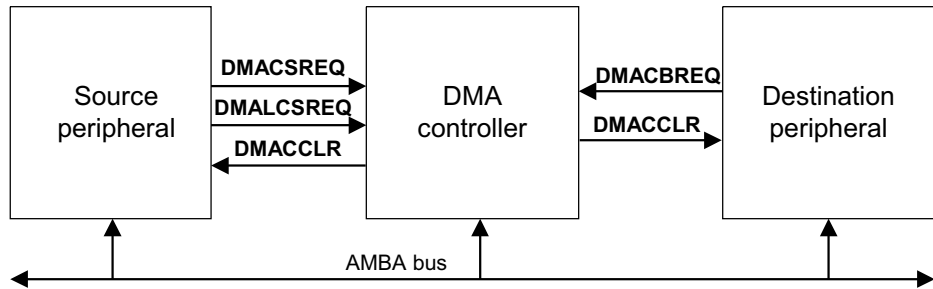


Figure B-15 Peripheral-to-peripheral transaction comprising single transfers

The source **DMACSREQ** and **DMALCSREQ** signals are mutually exclusive. The **DMALCSREQ** signal must be asserted to perform the last single transfer.

For transactions that use burst transfers and where the transaction is not a multiple of the burst size, use the following signals:

- the source single and burst request signals **DMACBREQ**, **DMACLBREQ**, **DMACSREQ**, and **DMALCSREQ**.
- the destination burst request signal, **DMACBREQ**.

This is shown in Figure B-16.

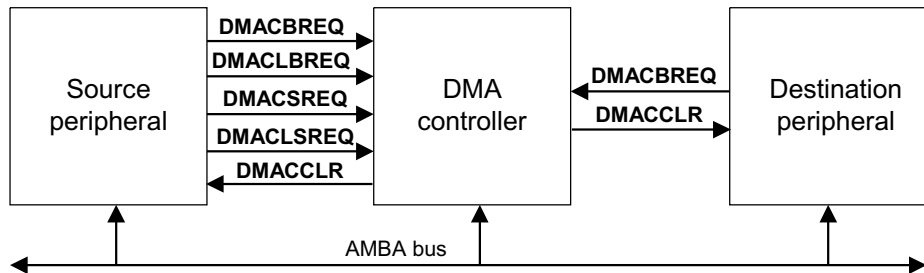


Figure B-16 Peripheral-to-peripheral transaction comprising bursts and single transfers

The PrimeCell SMDMAC has no knowledge of the length of the packet. Requests from the source peripheral are generated in the same way as described in *Peripheral-to-memory transactions under peripheral flow control* on page B-13. The PrimeCell SMDMAC initiates AHB reads (from the source peripheral to the PrimeCell SMDMAC internal FIFO) when requested, if there is space in the FIFO. When a last

request (**DMACLBREQ** or **DMACLSREQ**) is made, the appropriate read transfer is initiated and no more reads are performed until the next LLI is loaded. Writes from the PrimeCell SMDMAC FIFO to the destination peripheral are initiated when requested by the destination **DMACBREQ** if there is sufficient data in the FIFO. The PrimeCell SMDMAC is aware when the read operations have completed (as signaled by the source peripheral) and transfers any remaining data in the FIFO appropriately, using burst transfers of the defined burst length or less. When all the read and write transactions have completed the next LLI is loaded.

B.4.7 Peripheral-to-peripheral transactions under destination peripheral flow control

For transactions that are a multiple of the burst size, use the following signals:

- the source burst request **DMACBREQ**
- the single burst request **DMACSREQ** (if necessary)
- the destination burst request **DMACBREQ**
- the last burst request **DMACLBREQ**.

This is shown in Figure B-17.

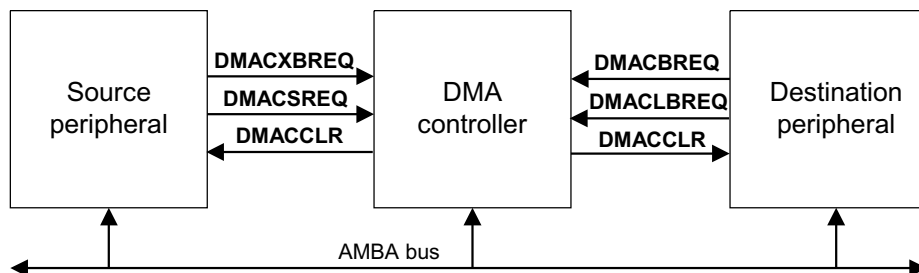


Figure B-17 Peripheral-to-peripheral transaction comprising bursts

The destination **DMACBREQ** and **DMACLBREQ** signals are mutually exclusive. The **DMACLBREQ** signal must be asserted to perform the last burst transfer.

For transactions comprising single transfers, use the following signals:

- the source single request signal **DMACSREQ**
- the single burst request, and **DMACBREQ**, (if necessary)
- the destination single request **DMACSREQ**
- the last single request **DMACLSREQ**.

This is shown in Figure B-18.

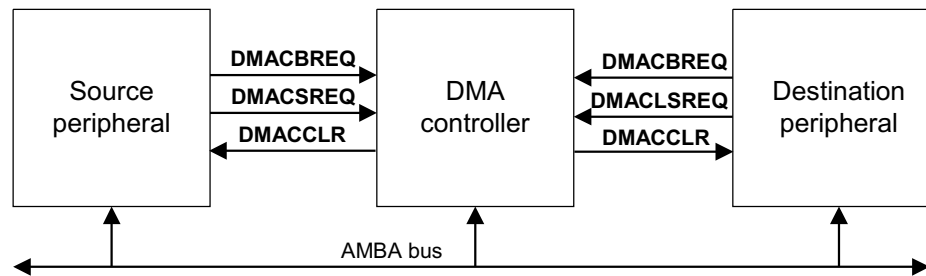


Figure B-18 Peripheral-to-peripheral transaction comprising single transfers

The destination **DMACSREQ** and **DMACLSREQ** signals are mutually exclusive. The **DMACLSREQ** signal must be asserted to perform the last single transfer.

For transactions that use burst transfers and where the transaction is not a multiple of the burst size, use the following signals:

- the source single request signal **DMACSREQ**
- the source burst request signal **DMACBREQ**
- the destination single request signals **DMACSREQ** and **DMACLSREQ**
- destination burst request signals **DMACBREQ** and **DMACLBREQ**.

This is shown in Figure B-19 on page B-19.

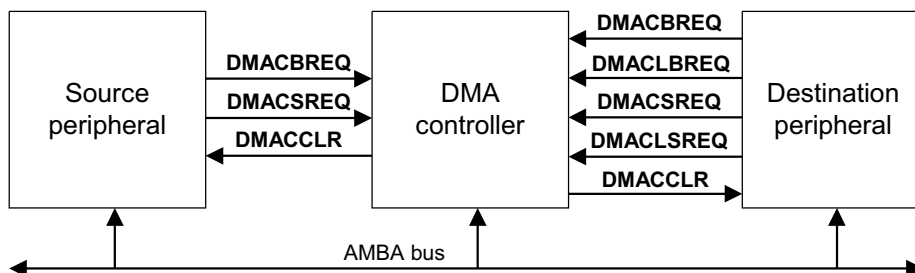


Figure B-19 Peripheral-to-peripheral transaction comprising bursts and single transfers

The PrimeCell SMDMAC has no knowledge of the length of the packet. Requests from the destination peripheral are generated exactly as described in *Memory-to-peripheral transaction under peripheral flow control* on page B-11.

When data is requested by destination peripheral, the PrimeCell SMDMAC transfers the required amount from the source peripheral as soon as the data is available. Data availability is signaled by **DMACBREQ** and **DMACSREQ** DMA requests from the source peripheral. When the destination peripheral indicates a last request (**DMACLBREQ** or **DMACLSREQ**), the PrimeCell SMDMAC transfers the required data from the source peripheral as soon as it is available. The PrimeCell SMDMAC then completes the write to the destination peripheral.

When all the read and write transactions have completed the next LLI is loaded.

Caution

If the destination peripheral width is smaller than the source peripheral width then care must be taken otherwise data can be lost at the end of a data transfer.

For peripheral-to-peripheral transfers with the destination as the flow controller, data is only transferred from the source peripheral when the destination peripheral requests it. If the source peripheral transfer width is 32-bits, and the destination peripheral transfer width is 8-bits, the following sequence can be envisaged:

1. Destination peripheral raises **DMACxSREQx** (one byte of data to be transferred).
2. Source peripheral raises **DMACxSREQx** and is serviced. This fetches one word data (four bytes).
3. One byte is transferred to destination peripheral.
4. Destination peripheral raises **DMACxLSREQx** (last single request).

5. One byte is transferred to the destination peripheral.
6. The transaction is now complete. Therefore out of the four bytes retrieved from the source peripheral, two are transferred to the destination peripheral, but two more are left in the channel FIFO in the PrimeCell SMDMAC. This data is then lost.

B.5 Signal timing

The timing behavior of the DMA signals is as follows:

DMA request signal **DMAC{L}(B/S)REQx**

Informs the PrimeCell SMDMAC that a peripheral is ready to proceed with a DMA transfer of the indicated size.

Active HIGH. Sampled by the PrimeCell SMDMAC on the positive edge of **HCLK**. The DMA request signals are used in conjunction with the **DMACCLR** signal to perform handshaking.

DMA Acknowledge or Clear **DMACCLR**x

Indicates to the slave that a DMA transfer has completed.

Active HIGH.

DMA Terminal Count **DMACTC**x

Indicates to the slave that the end of packet has been reached.

Active HIGH.

————— **Note** —————

If the DMA request source does not use the same clock as the PrimeCell SMDMAC, then the request must be synchronized by setting the relevant bit in the **DMACSync** register.

B.6 Functional timing diagram

A peripheral asserts a DMA request and holds it active. The **DMACCLR** signal is asserted by the PrimeCell SMDMAC when the last data item has been transferred. When the peripheral sees that the **DMACCLR** signal has gone active it takes the DMA request signal inactive. The PrimeCell DMAC controller deasserts the **DMACLR** signal when the DMA request signal goes inactive.

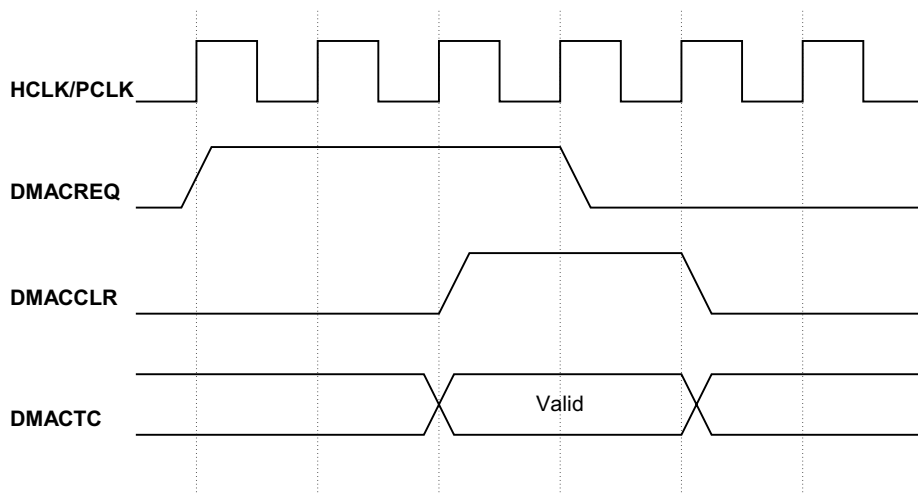


Figure B-20 DMA interface timing

B.7 PrimeCell SMDMAC transfer timing diagram

Figure B-21 shows the state of the PrimeCell SMDMAC response and request signals, AHB interface signals and interrupt request signals for a complete DMA transfer.

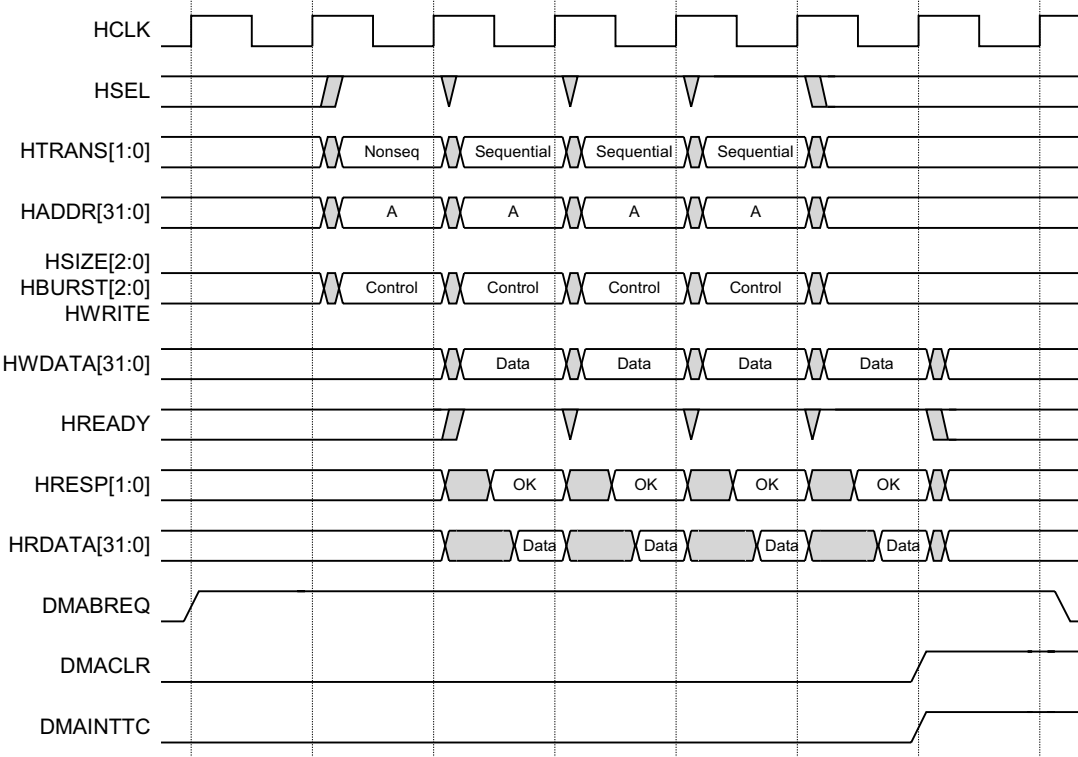


Figure B-21 PrimeCell SMDMAC transfer timing diagram

Appendix C

Scatter/Gather

This section describes scatter/gather through LLI. It contains the following section:

- *Scatter/gather through linked list operation* on page C-2.

C.1 Scatter/gather through linked list operation

The source and destination data areas are defined by a series of linked lists. Each LLI controls the transfer of one block of data, and then optionally loads a further LLI to continue the DMA operation, or stops the DMA stream. The first *Linked List Item* (LLI) is programmed into the PrimeCell SMDMAC

The data to be transferred described by a LLI (referred to as the packet of data) usually requires one or more DMA bursts (to each of the source and destination).

See Figure C-1 on page C-2 for an example of a LLI. A rectangle of memory has to be transferred to a peripheral. The addresses of each line of data is given (in hexadecimal) at the left-hand side of the figure. The LLIs describing the transfer are to be stored contiguously from address 0x20000.

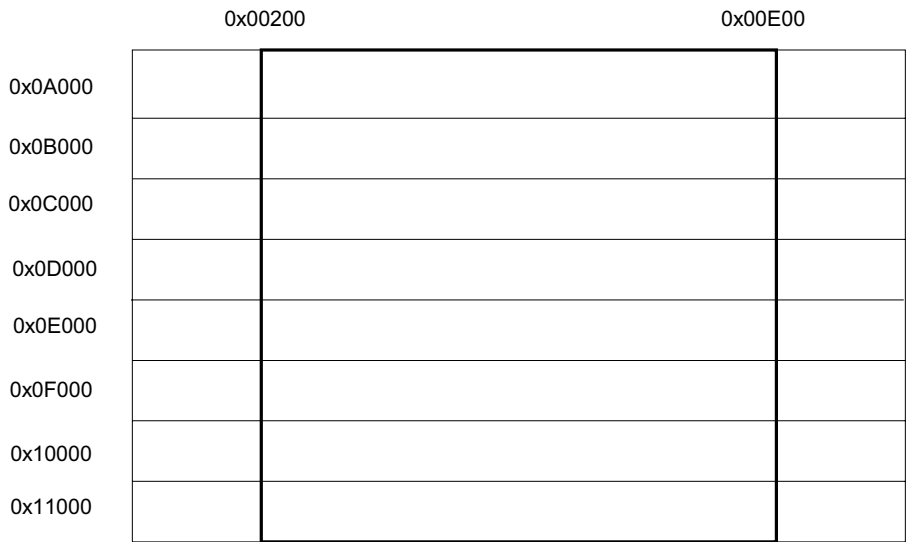


Figure C-1 LLI example

The first LLI, stored at 0x20000, defines the first block of data to be transferred, which is the data stored between addresses 0x0A200 and 0x0AE00:

- source start address 0x0A200
- destination address set to the destination peripheral address
- transfer width, word (32-bit)
- transfer size, 3072 bytes (0xC00)
- source and destination burst sizes, 16 transfers

- next LLI address, 0x20010

The second LLI, stored at 0x20010 describes the next block of data to be transferred:

- source start address 0x0B200
- destination address set to the destination peripheral address.
- transfer width, word (32-bit)
- transfer size, 3072 bytes (0xC00)
- source and destination burst sizes, 16 transfers
- next LLI address, 0x20020.

A chain of descriptors is built up, each one pointing to the next in the series. To initialize the DMA stream, the first LLI (0x20000) is programmed into the PrimeCell SMDMAC. When the first packet of data has been transferred the next linked list item is automatically loaded.

The final LLI is be stored at 0x20070 and contains:

- source start address 0x11200
- destination address set to the destination peripheral address
- transfer width, word (32-bit)
- transfer size, 3072 bytes (0xC00)
- source and destination burst sizes, 16 transfers
- next LLI address, 0x0.

Since the next LLI address is set to zero, this is the last descriptor, and the DMA channel is disabled after transferring the last item of data. The channel is probably set to generate an interrupt at this point to indicate to the ARM processor that the channel can be reprogrammed.

Index

The items in this index are listed in alphabetical order, with symbols and numerics appearing at the end. The references given are to page numbers.

D

DMA interface A-1
DMA request and response signals A-3

F

Functional blocks 2-1

I

Interrupt generation logic 2-10
Interrupt request signals A-2

L

Logic
 interrupt generation 2-10
 test 2-6

M

Maskable interrupts 2-10

P

PrimeCell DMA controller
 block diagram 2-3
 features 1-2
 functional blocks 2-1
 register descriptions 3-10
 signal descriptions A-1

R

Register block 2-4
Register descriptions 3-10
Registers
 DMAConfiguration 3-15, 3-16
 DMACPeriphID0-3 3-26
 DMAC0Configuration 3-23

DMAC0Control 3-19
DMAC0DestAddr 3-18
DMAC0LLI 3-18
DMAC0SrcAddr 3-17
DMAEnabledChannels 3-13
DMAIntErrorStatus 3-11, 3-12
DMAIntStatus 3-10
DMARawIntErrorStatus 3-13
DMARawIntTCStatus 3-12
DMASoftBReq 3-13
DMASoftSReq 3-11, 3-14, 3-15

S

Scatter/gather C-1
Signal descriptions A-1
Signals
 AHB master A-6
 AHB master bus request A-8
 AHB slave A-4
 scan test control A-9

T

Test logic 2-6

The DMARawIntTCStatus 3-12